Leveraging Imperfect-Orthogonality Aware Scheduling for High Scalability in LPWAN

Zhuqing Xu, Junzhou Luo, *Member, IEEE,* Zhimeng Yin, *Member, IEEE,* Shuai Wang, *Member, IEEE,* Ciyuan Chen, Jingkai Lin, Runqun Xiong, *Member, IEEE,* and Tian He, *Fellow, IEEE*

Abstract—As an emerging Low-Power Wide Area Networks (LPWAN) technology, LoRa is dedicated to providing long-range connections for pervasive Internet-of-Things devices. As LoRa operates in the unlicensed spectrum with an ALOHA-based MAC-layer protocol stack, transmissions from multiple LoRa end-devices inevitably collide with each other, leading to packet losses and increased transmission delay. Targeting at collisions caused by interferences under the *same* spreading factor (SF) settings, researchers introduce multiple lines of techniques. Despite their efforts, these techniques commonly neglect the potential collisions caused by interferences under *different* SF settings, resulting in imperfect orthogonality. Given the disparate transmission power configurations and diverse deployed locations, the collisions under different SFs commonly exist in practical networks and significantly limit the LoRa reliability. This paper presents X-MAC, the first scheduler aware of imperfect orthogonality. Technically, X-MAC detects the collisions under different SFs via tracking historical transmissions, and performs dynamic channel scheduling to avoid collisions caused by interferences under the same and different SFs. Extensive evaluations on testbed devices show that, compared with the state-of-the-art methods, X-MAC boosts the network scalability (number of concurrent end-devices) by 1.26$\times$ to 2.41$\times$ with packet reception rate requirement of $>$ 95%.

Index Terms—LPWAN, LoRa, high scalability, imperfect-orthogonality, scheduling

1 Introduction

In recent years we have witnessed the proliferation of emerging Low-Power Wide Area Networks (LPWAN). Among existing LPWAN technologies, LoRa is a widely adopted solution, capturing considerable attention from scientific and industrial communities. Since LoRa offers communication at a long distance, e.g., 10km, a LoRa network potentially covers a large area, calling for high scalability. For this purpose, LoRa introduces chirp spread spectrum (CSS) and assigns different spreading factors (SF) to end-devices at different distances. When end-devices transmit with different SF parameters, CSS is possible to offer additional resilience for alleviating transmission corruptions.

To further improve the scalability, researchers introduce various types of solutions. Based on the design principles, the existing literature is summarized into the following two categories: PHY-layer approaches and MAC-layer approaches. PHY-layer approaches (e.g., Choir [1], FTrack [2], and CoLoRa [3]) examine the signal characteristics from different LoRa devices, so that they could resolve collided transmissions. However, these techniques require direct access to PHY samples and are not feasible on commodity chips. In contrast, MAC-layer approaches (e.g., S-MAC [4], LMAC [5], and others [6], [7], [8], [9], [10], [11]) adjust the frequency channel, transmission timing, or power of LoRa end-devices that have the same SF. By doing so, these techniques effectively avoid transmission collisions under the same SF (termed as collisions caused by co-SF interferences) for improving the communication resilience and are directly applicable on commodity LoRa chips.

Although existing literature offers good network scalability, they focus on addressing collisions caused by co-SF interferences, since it was commonly believed that LoRa transmissions with different SFs are resilient to each other. However, recent studies [12], [13], [14], [15], [16] find that LoRa symbols with different SFs are quasi-orthogonal (imperfectly orthogonal), thus could also lead to potential transmission collisions (termed as collisions caused by inter-SF interferences). This problem is especially critical when LoRa end-devices have different deployment locations and signal powers, which is general in practical LoRa networks. Since strong signal receptions from nearby transmitters could overwhelm weaker from faraway transmitters [1], [2], [3], and it is pretty common for end-devices to be spread over a wide area [16], [17], [18], [19], [20], [21], such as in the smart city, smart agriculture, and industrial monitoring, etc.

To improve the network scalability, this paper proposes X-MAC, the first scheduler that considers imperfect orthogonality in practical LoRa networks. Specifically, X-MAC collects the inter-SF interferences by tracking the successful and failed transmissions. Then it solves an optimization problem to assign a frequency channel (and a transmission timing) to each packet to be transmitted. If the inter-SF/co-SF interference between two LoRa packets causes a transmission collision, X-MAC will consider assigning different frequency channels (or transmission timings) so that their transmissions will not collide with each other. X-MAC further proposes the corresponding imperfect-orthogonality

- *Zhuqing Xu is with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing 211106, China. E-mail: zqxu@seu.edu.cn.*
- *Junzhou Luo, Shuai Wang, Ciyuan Chen, Jingkai Lin, Runqun Xiong and Tian He are with the School of Computer Science and Engineering, Southeast University, Nanjing 210096, China. E-mail: {jluo, shuaiwang, cychen, jklin, rxiong, tianhe}@seu.edu.cn.*
- *Zhimeng Yin is with the Department of Computer Science, City University of Hong Kong, Hong Kong, China. E-mail: zhimeyin@cityu.edu.hk.*

Manuscript received 23 Apr. 2023; revised 24 Dec. 2023.
(Corresponding authors: Runqun Xiong.)

awareness and channel scheduling mechanisms for multi-gateway network scenarios. It also offers additional designs, such as closed-loop adaptive status update, clock drift correction, and downlink gateway selection, to enable a practical MAC protocol in LoRa networks.

To summarize, X-MAC has the following contributions:

- We design X-MAC, a MAC-layer scheduler that is aware of the imperfect orthogonality for improving the network scalability. To the best of our knowledge, X-MAC is the first MAC protocol that considers imperfect orthogonality in practical LoRa deployment, in addition to the commonly considered collisions caused by co-SF interferences.
- We introduce channel scheduling optimization models for single-gateway and multi-gateway network scenarios respectively, followed by an imperfect-orthogonality aware scheme to detect transmission collisions caused by inter-SF interferences, and the effective channel scheduling solutions based on this to decide the frequency channel (and transmission timing) of each packet to be transmitted.
- We implement X-MAC on a real-world testbed and a simulation platform for extensive evaluations of its performance and cost under various scenarios. The results show that, compared with the state-of-the-art methods, X-MAC boosts network scalability (number of concurrent end-devices) by 1.26× to 2.41× with packet reception rate requirement of $> 95\%$.

2 Background and Motivation

2.1 LoRa Background

LoRaWAN is an ALOHA (i.e., random access) based MAC-layer protocol stack. The network architecture proposed by LoRaWAN specification [22] consists of end-devices (EDs), gateways (GWs), a network server (NS), and an application server. End-devices and a gateway are connected in a single-hop star topology. The gateway handles receiving and forwarding packets to the network server over an IP-based network. The network server is in charge of address checking, authentication, de-duplication, and link configuration, etc.

In LoRa networks, carrier frequency (CF) and spreading factor (SF) are two essential transmission settings. Specifically, CF is the central transmission frequency of a LoRa packet. SF decides the ratio between the modulation bandwidth BW and symbol rate R_s by $SF = \log_2(BW/R_s)$. A higher SF offers a longer transmission duration for a LoRa symbol, thus increasing its resilience against wireless interference and background noises. This increases the communication range but also leads to higher energy consumption.

LoRaWAN supports a SF choice from 7 to 12. If time-overlapped LoRa packets use the same CF and SF respectively, their transmissions will lead to **co-SF interference**. When the gateway demodulates these signals, the FFT results will produce several indiscernible peaks. Thus, only the packet with the strongest signal power is demodulated if its power is at least ~ 4 times (6 dB) [21], [23] stronger than others. To increase the capacity of LoRa networks, recent studies, e.g., S-MAC [4] and LMAC [5], adjust the channels or transmission timing of packets via scheduling or channel activity detection, so that transmissions with the same SF are collision-free.

2.2 Inter-SF Interference

In addition to co-SF interference, **inter-SF interference** occur between time-overlapped packets with the same CF but different SF parameters. This is because, with the CSS, LoRa symbols with different SFs are quasi-orthogonal (imperfectly orthogonal) [12], [13], [14], [15], [16], instead of perfectly orthogonal. As a result, ongoing LoRa transmissions will inevitably lead to wireless interference among each other, and the inter-SF capture condition must be met to demodulate the desired signal from the interfering signals [12], [13], [14], [16], which is given by:

$$SINR_{dsr} = \frac{P_{dsr}}{P_{int} + \sigma^2} \geq \theta_{SF_{dsr}} \quad (1)$$

where P_{dsr} refers to the received power of the desired signal, P_{int} is the received total power of the interfering signals using different SFs, and σ^2 is the power of additive white Gaussian noise (AWGN) of the wireless channel. If the signal-to-interference-plus-noise ratio (SINR) of the desired packet exceeds the corresponding threshold $\theta_{SF_{dsr}}$, then this packet could be correctly demodulated.

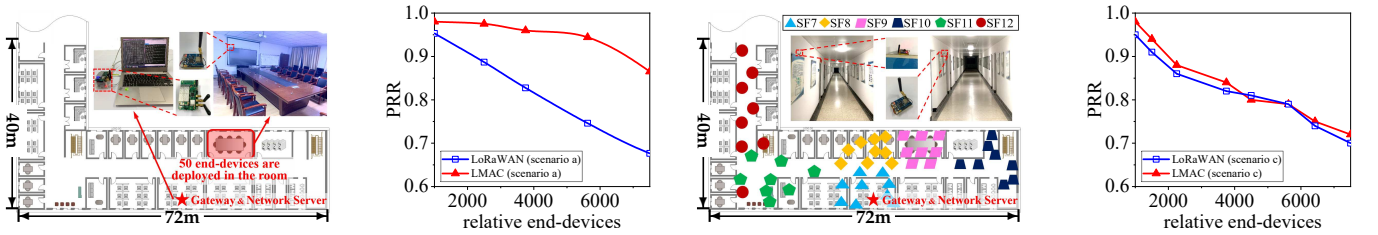
Tracing the root, when demodulating the desired signal, the CSS demodulator treats other symbols with different SFs as modulated noise. If the amplitudes of two signals differ significantly, correlation peaks can appear at different positions from their symbol time, and demodulation error occurs.

For simplicity, we refer to any packet reception failure caused by co-SF interference as the **co-SF collision**, and any packet reception failure caused by inter-SF interference as the **inter-SF collision**.

2.3 Limitations of State-of-the-art

Suppose that the distance of an end-device using SF12 to the gateway is 6 times that of an end-device using SF7 to the gateway. With log-distance path loss model [18], [19], [24], it gives the end-device using SF7 a 23 to 31 dB power margin over the end-device using SF12 (for path loss exponents of 3, 4, resp.). According to demodulation thresholds of inter-SF interference [13], [16], [25], it can be theoretically confirmed that any transmission with SF7 will at least destroy the concurrent transmissions with SF12 (and most probably SF11).

To examine these different types of interferences in actual LoRa networks and their impacts, we first conduct the following series of experiments in an indoor office shown in Fig. 1(a). In this scenario, all 50 end-devices are deployed in a room, and have the same SF parameter, e.g., $SF = 9$. Due to the difficulty of deploying a large-scale LoRa network in practice [19], we adopt a common emulation solution, following [4], [20], which changes the transmission periods of end-devices to emulate a larger network. Fig. 1(b) shows the packet reception rate comparison between native LoRaWAN, a pure ALOHA-based solution, and the state-of-the-art co-SF MAC, LMAC [5], which utilizes channel activity detection (CAD) to avoid collisions among LoRa transmissions with the same SF. It can be found that the



(a) Centralized deployment scenario where all 50 end-devices use SF9 (b) PRR vs. relative end-devices under scenario (a) (c) Decentralized deployment scenario where 50 end-devices use different SFs (d) PRR vs. relative end-devices under scenario (c)

Fig. 1. The packet reception rate (PRR) of the native LoRaWAN and LMAC in different scenarios. Fig. 1(a) and 1(c) show the centralized and decentralized deployment scenarios, resp. Fig. 1(b) and 1(d) show the corresponding PRR results, resp. The bandwidth, coding rate, packet length, transmission power are set to 125 kHz, 4/5, 40 bytes, and 14 dBm, resp. The detailed setup is in Section 7.1.

performance of LMAC significantly outperforms LoRaWAN under various network scales.

However, in typical LoRa networks, different LoRa end-devices are deployed in various locations, thus resulting in different received powers at the gateway and further leading to different choices of SF for ensuring communication resilience. Fig. 1(c) depicts the same indoor office as Fig. 1(a), but 50 end-devices are decentralized deployed and their SF parameters are decided by specific channel fading. Fig. 1(d) shows that the co-SF MAC (LMAC[1]) suffers from heavy packet losses and is only slightly better than the native LoRaWAN. This is because the channel activity detection (CAD) employed by LMAC is SF-selective, i.e., it can only detect transmissions in a pre-specified SF, while remaining insensitive to other transmissions with different SFs. This experiment demonstrates the severe impacts of collisions due to imperfect orthogonality, which is commonly neglected in current solutions.

Overall, considering the high density and wide coverage of end-devices, in practical LoRa networks, it is common that end-devices with different SF settings are spread over a wide geographical area [1], [2], [3], [16], [17], [18], [19], [20]. As a result, the inter-SF interferences significantly affect the communication reliability and impose challenges for improving the scalability of LoRa networks, calling for specific solutions.

3 Overview of X-MAC

To the best of our knowledge, X-MAC is the first MAC-layer scheduler that considers the imperfect orthogonality (inter-SF collisions) for improving the scalability in practical LoRa networks. Fig. 2 illustrates its architecture.

End-devices report their transmission periods in their joining request in X-MAC.[2] To be compatible with the native LoRaWAN protocol, X-MAC preserves LoRaWAN packet structure, and fills reserved fields in the packet header for the additional control information, e.g., identify and confirm the relevant status. For each uplink packet to be transmitted, the end-device configures the specified channel configuration information (CCI), i.e., frequency channel and transmission timing, which is pre-assigned by the network server operating our X-MAC scheduling. For each received

1. The author opened the source code at https://wands.sg/lmac/. We use the best configuration "Level 3" provided by the website for experiments.

2. We assume that the transmission of end-devices is periodic, as detailed in the first assumption in Section 4.1.

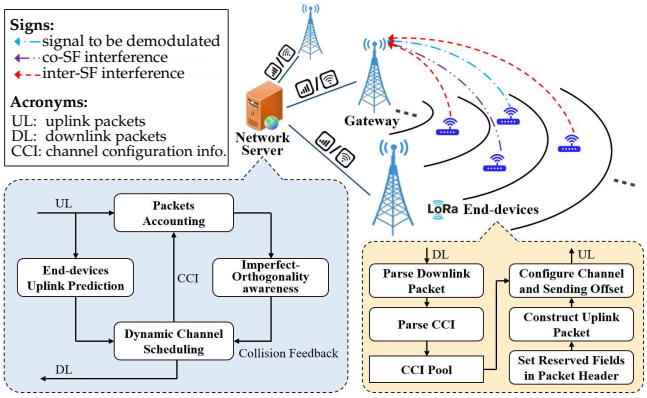


Fig. 2. The architecture of the X-MAC.

downlink packet from its receive window, the end-device parses and saves the CCI (if any) carried in it.

The network server operates the following additional functions for improving the network scalability: First, the imperfect-orthogonality aware scheme collects inter-SF collisions by tracking the successful and failed transmissions. Second, taking the detected inter-SF collision information, the dynamic channel scheduler configures a channel for each transmission to avoid co-SF collisions and currently detected inter-SF collisions. Then it constructs the CCI required by an end-device, and distributes to its receive window, either as a separate downlink message or by attaching it to the application's own downlink message (the piggyback way). In addition, by analyzing the status information (e.g., pairs of the packet counter and the corresponding receiving time) of the received packets, the network server corrects the clock drift of each end-device, and predicts the transmission time of subsequent uplink packets (detailed in Section 6.2).

4 Core Design

Depending on the required network coverage, deployment and maintenance cost, etc., many small-scale application scenarios deploy only one gateway for an entire LoRa network. Moreover, as can be observed from Fig. 1, the transmission collisions caused by the imperfect orthogonality of SFs are also extremely serious in these single-gateway networks. Thus, this section is dedicated to considering the single-gateway network scenarios.

4.1 System Model

In practical LoRa single-gateway networks, massive devices are deployed in different locations, requiring varying SF

settings depending on the specific channel status. Accordingly, the core of X-MAC is to capture inter-SF collisions without much prior knowledge, and further schedule each transmission of end-devices to avoid packet corruption.

Assumptions.

- The end-devices' transmissions are periodic, while their periods are different depending on specific application settings. This is primarily inspired by the observation that, in many application scenarios, such as metering, monitoring, and tracking, the traffic of end-devices has periodic characteristics [4], [26]. Moreover, according to the analysis of a massive volume of data captured from the world-wide LoRaWAN network platform The Things Network (TTN) [27], it is found that $>$ 93% LoRa end-devices are periodic [28], [29]. Thus, this paper mainly studies scenarios with periodic transmission characteristics.
- The network server determines the CF of a LoRa uplink transmission, while SF is disabled to adjust arbitrarily. Since in native LoRaWAN, end-devices can inherently randomly select a CF from the available channels for each transmission. On the other hand, SF is closely related to the range, data rate and energy consumption of an end-device, and modifying it arbitrarily may break application constraints on delay, energy, etc. Further, X-MAC can already achieve high scalability and reliability, without modifying SF.
- The gateway is full-duplex that supports simultaneous transmitting and receiving packets. Since LoRaWAN regional parameters specification [30] has offered separate uplink and downlink channels for most regulatory regions worldwide. In addition, it is feasible and low-cost to implement such a full-duplex gateway by using a separate RF chip for downlink transmission, and Semtech also introduces a full-duplex gateway reference design [31].

Fig. 3 depicts a scenario of strict periodic transmissions from end-devices. Each end-device has a transmission period. The notations s_i, f_i denote the transmission start, finish time of packet i, resp. The end-device transmits a packet immediately at the beginning of a transmission period, which is termed as "*strict*". A packet transmission from s_i to f_i is hereinafter referred to as a *transmission event*, or simply an *event*. Table 1 summarizes the primary notations.

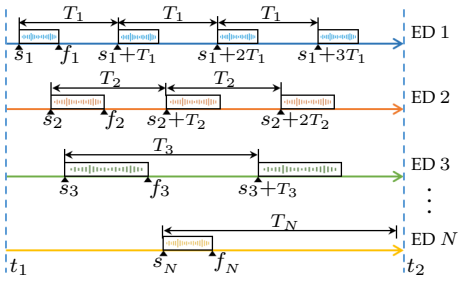


Fig. 3. The scenario of strict periodic transmissions from end-devices.

To improve the scalability, we offer the following optimization problem by considering imperfect orthogonality.

$$\max_{\boldsymbol{x}_i} \sum\nolimits_{i=1}^{n} \mathbf{1}^T \boldsymbol{x}_i (1 - \iota^i_{CL}(E \setminus \{e_i\})) \tag{2}$$

TABLE 1
Important notations and descriptions

Notation	Description
n	Number of transmission events, where $n \in \mathbb{Z}_{++}$.
m	Number of frequency channels, where $m \in \mathbb{Z}_{++}$.
i, j	Index of event, where $i, j \in \mathbb{Z}_n$, $\mathbb{Z}_n = \{1, ..., n\}$.
k	Index of CF, where $k \in \mathbb{Z}_m$, $\mathbb{Z}_m = \{1, ..., m\}$.
E	Set of total events, where $E = \{e_1, ..., e_n\}$.
U, V	Non-empty subset of E, where $U, V \neq \varnothing$, $V \subseteq U \subseteq E$.
e_i	The i-th event, where $e_i \in E$.
s_i	Transmission start time of e_i, where $s_i \in \mathbb{R}_+$.
f_i	Transmission finish time of e_i, where $f_i \in \mathbb{R}_+$, $s_i < f_i$.
$\boldsymbol{C}$	c_{ij} in $\boldsymbol{C} = [\boldsymbol{c}_1\ \boldsymbol{c}_2\ ...\ \boldsymbol{c}_n] \in \mathbb{S}^n$ denotes whether a collision will occur between e_i and e_j, where $c_{ij} \in \{0, 1\}$.
$\boldsymbol{T}$	t_{ij} in $\boldsymbol{T} = [\boldsymbol{t}_1\ \boldsymbol{t}_2\ ...\ \boldsymbol{t}_n] \in \mathbb{S}^n$ denotes whether e_i, e_j overlap in time, where $t_{ij} \in \{0, 1\}$. If $i \neq j$, $s_i \leq f_j$, $f_i \geq s_j$, then let $t_{ij} = 1$; else $t_{ij} = 0$.
$\boldsymbol{P}$	p_{ij} in $\boldsymbol{P} = [\boldsymbol{p}_1\ \boldsymbol{p}_2\ ...\ \boldsymbol{p}_n] \in \mathbb{S}^n$ denotes whether e_i, e_j use the same SF, where $p_{ij} \in \{0, 1\}$.
$\boldsymbol{R}$	r_{ij} in $\boldsymbol{R} = [\boldsymbol{r}_1\ \boldsymbol{r}_2\ ...\ \boldsymbol{r}_n] \in \mathbb{S}^n$ denotes whether a collision caused by the inter-SF interference will occur between e_i and e_j, when $t_{ij} = 1$, $x_i^T x_j = 1$, $p_{ij} = 0$. $r_{ij} \in \{0, 1\}$.
$\boldsymbol{X}$	x_{ki} in $\boldsymbol{X} = [\boldsymbol{x}_1\ \boldsymbol{x}_2\ ...\ \boldsymbol{x}_n] \in \mathbb{R}^{m \times n}$ denotes whether CF k is used by e_i, where $x_{ki} \in \{0, 1\}$.

Subject to:

$$\iota^i_{CL}(U) := \begin{cases} 1, & \exists V \subseteq U,\ \iota^i_{OL}(V)\iota^i_{CF}(V)\iota^i_{SF}(V) \neq 0, \text{ or} \\ & \iota^i_{OL}(V)\iota^i_{CF}(V)(1-\iota^i_{SF}(V))\iota^i_R(V) \neq 0 \\ 0, & \text{otherwise} \end{cases} \tag{3}$$

$$\mathbf{1}^T \boldsymbol{x}_i \leq 1,\ \forall i \in \mathbb{Z}_n \tag{4}$$

$$x_{ki} \in \{0, 1\},\ \forall i \in \mathbb{Z}_n,\ \forall k \in \mathbb{Z}_m \tag{5}$$

where function $\iota^i_{OL}(V)$ is a 0-1 indicator: $\iota^i_{OL}(V) = 1$ only if $\forall e_j \in V,\ t_{ij} = 1$. Function $\iota^i_{CF}(V)$ is a 0-1 indicator: $\iota^i_{CF}(V) = 1$ only if $\forall e_j \in V,\ \boldsymbol{x}_i^T \boldsymbol{x}_j = 1$. Function $\iota^i_{SF}(V)$ is a 0-1 indicator: $\iota^i_{SF}(V) = 1$ only if $\forall e_j \in V,\ p_{ij} = 1$. Function $\iota^i_R(V)$ is a 0-1 indicator: $\iota^i_R(V) = 1$ only if $\iota^i_{OL}(V) = 1$, $\iota^i_{CF}(V) = 1$, $\iota^i_{SF}(V) = 0$, and e_i is collided by events in V.

The objective (2) is to assign an appropriate frequency channel for each event, such that the number of successfully delivered packets is maximized. The 0-1 indicator function (3) defines whether e_i has a transmission collision (co-SF/inter-SF collision) with the events in U, where $\iota^i_{CL}(U) = 0$ only if e_i does not collide with any subset events in U. The variable x_{ki} in (5) denotes whether e_i uses channel k. The Ineq. (4) denotes that for any e_i, it can only select at most one frequency channel for transmission. The vector $\boldsymbol{x}_i = \mathbf{0}$ means e_i is not allowed to be transmitted.

However, it is challenging in practical LoRa networks to fully acquire inter-SF collision information for all subsets of events, i.e., $\iota^i_R(V)$, for $\forall V \subseteq (E \setminus \{e_i\})$, $i \in \mathbb{Z}_n$, since their scale reaches $O(n2^n)$. Inspired by the following key insight, we can simplify the scale of the required inter-SF collision information to $O(n^2)$ using the idea of iterative dimensionality reduction. That is, only inter-sf collision information between any two events is required.

Suppose that an inter-SF collision occurs when n ($n \geq 3$) events (e.g., $\{e_1, e_2, e_3\}$) are transmitted simultaneously. If we determine that the inter-SF collision occurs between 2 events (e.g., e_1 and e_2) in n, then with the ensuing rational channel scheduling, the collision between these 2 events is avoided, i.e., for these n events, the inter-SF collision can only appear in $(n-1)$ events (e.g., $\{e_1, e_3\}$, or $\{e_2, e_3\}$). The rest may be deduced by analogy. To this end, by well identifying the inter-SF collision between each pair of events and

rational channel scheduling, the inter-SF collision among multiple events can be effectively handled. Accordingly, the problem is updated as follows.

$$\max_{\boldsymbol{x}_i} \sum_{i=1}^{n} \mathbf{1}^T \boldsymbol{x}_i \prod_{j=1, j\neq i}^{n} (1 - c_{ij}) \tag{6}$$

Subject to:

$$\boldsymbol{c}_i = \boldsymbol{t}_i \circ (\boldsymbol{X}^T \boldsymbol{x}_i) \circ \boldsymbol{p}_i + \boldsymbol{t}_i \circ (\boldsymbol{X}^T \boldsymbol{x}_i) \circ (\mathbf{1} - \boldsymbol{p}_i) \circ \boldsymbol{r}_i \tag{7}$$

$$\mathbf{1}^T \boldsymbol{x}_i \leq 1,\ \forall i \in \mathbb{Z}_n \tag{8}$$

$$x_{ki} \in \{0, 1\},\ \forall i \in \mathbb{Z}_n,\ \forall k \in \mathbb{Z}_m \tag{9}$$

where the symbol $\circ$ denotes the *Hadamard product*. The vector $\boldsymbol{c}_i$ in Eq. (7) is the i-th column in matrix $\boldsymbol{C}$, and each component c_{ij} in $\boldsymbol{c}_i$ indicates whether a collision (co-SF or inter-SF collision) occurs between e_i and e_j.

To tackle this problem, the matrix $\boldsymbol{R}$ (explained in Table 1) used in Eq. (7) requires to be determined primarily. Thus, we first design an imperfect-orthogonality aware scheme to detect inter-SF collisions. Then, a performance-bounded channel scheduling solution for the above optimization is proposed to maximize packet delivery. Besides, additional designs are offered to make it practical in LoRa networks.

4.2 Imperfect-Orthogonality Awareness

To detect inter-SF collisions, the first intuition is to model the propagation channel by theoretical analysis [12], [13], [32] or learning-based approaches [23], [24]. However, inter-SF collisions are closely related to the deployment sites of networks, ambient environment, and signal power from other end-devices, etc., making it challenging for them to adapt to various environments and avoid such collisions effectively. On the other hand, these methods require rich prior knowledge, such as deployment locations, transmission parameters, multi-spectral images, etc., which greatly increases the practical deployment complexity.

To this end, we draw inspiration from the observation that inter-SF collision information can be identified by tracking the expected and actual packet reception results of channel scheduling. The key idea of the proposed practical imperfect-orthogonality aware scheme is to check the reception of packets that meet the precondition of inter-SF collisions. That is, for the case where two end-devices transmit overlapped packets with the same CF and different SFs, if packets are lost frequently, it is considered that there is an inter-SF collision with high probability. Otherwise, it is inferred they are collision-free.

In light of this, we design the imperfect-orthogonality aware scheme (Algorithm 1), which utilizes the expected received packets of channel scheduling, and the corresponding actually received packets to detect inter-SF collisions (i.e., acquire $\boldsymbol{R}$). Then the updated $\boldsymbol{R}$ is further employed as an input of the next round of channel scheduling (Section 4.3 or 4.4). The entire process is performed alternately and iteratively (Section 6.1). As shown in Algorithm 1, the notations E_{ps}, E_{cs} denote the event sets in the past and future time intervals, resp. d indicates the number of running rounds, and the duration of a round is denoted by the variable T_{next} in line 4 of Algorithm 7, that is, a multiple of the maximum transmission period in the currently joined end-devices (detailed in Section 6.1). $\boldsymbol{Q} \in \mathbb{S}^N$ indicates the number of inter-SF collisions between end-devices, where N is the number of joined end-devices. It is initially a zero matrix and is updated in each round d. The constant γ denotes the confidence threshold of an inter-SF collision. In the implementation of X-MAC, we set γ to 3 to balance collision detection efficiency, performance, and stability. It is evaluated and elaborated in Sections 7.7 and 7.8.

Algorithm 1 Imperfect-Orthogonality Aware for Single-GW

1: **input:** E_{ps}, E_{cs}, $\boldsymbol{Q}^d$
2: **output:** $\boldsymbol{Q}^{d+1}$, $\boldsymbol{R}$
3: $E_{dl} \leftarrow \{e_j \mid e_j \in E_{ps}, e_j$ was sent according to the CCI$\}$
4: $E_{rx} \leftarrow \{e_j \mid e_j \in E_{ps}, e_j$ was received by the gateway$\}$
5: **for** each $e_i \in (E_{dl} \setminus E_{rx})$ **do**
6: $\quad E_{cn} \leftarrow \{e_j \mid e_j \in E_{ps},\ t_{ij} = 1,\ \boldsymbol{x}_i^T \boldsymbol{x}_j = 1,\ p_{ij} = 0\}$
7: $\quad e_j \leftarrow$ event with max RSSI in $(E_{cn} \cap E_{rx})$
8: $\quad$ **if** $e_j = \varnothing$ **then**
9: $\quad\quad e_j \leftarrow$event with max history avg. RSSI in $(E_{cn} \cap E_{dl})$
10: $\quad$ **end if**
11: $\quad$ Get indexes x, y of EDs to which e_i, e_j belong, resp.
12: $\quad q_{xy} \leftarrow q_{xy} + 1$; $q_{yx} \leftarrow q_{yx} + 1$
13: **end for**
14: **for** each $e_i \in E_{rx}$ **do**
15: $\quad E_{cn} \leftarrow \{e_j \mid e_j \in E_{ps},\ t_{ij} = 1,\ \boldsymbol{x}_i^T \boldsymbol{x}_j = 1,\ p_{ij} = 0\}$
16: $\quad e_j \leftarrow$ event with min RSSI in $(E_{cn} \cap E_{rx})$
17: $\quad$ Get indexes x, y of EDs to which e_i, e_j belong, resp.
18: $\quad$ **if** $q_{xy} > 0$ **then** $q_{xy} \leftarrow q_{xy} - 1$; $q_{yx} \leftarrow q_{yx} - 1$
19: **end for**
20: **for** any $e_i,\ e_j \in E_{cs}$ **do**
21: $\quad$ Get indexes x, y of EDs to which e_i, e_j belong, resp.
22: $\quad$ **if** $q_{xy} \geq \gamma$ **then** $r_{ij} \leftarrow 1$ **else** $r_{ij} \leftarrow 0$
23: **end for**

Specifically, for each unexpected lost packet e_i that was sent according to the CCI but not received by the gateway, find the interfering packet e_j most probably to cause its loss (lines 6-10), which provides the most interfering power (refer to the demodulation condition in Eq. (1)), and increase the number of inter-SF collisions between the transmitters of e_i, e_j by 1 (lines 11-12). On the other hand, for each received packet, find the interfering packet that has the least impact on its reception, and decrease the number of corresponding inter-SF collisions by 1 (lines 15-18). With these, $\boldsymbol{R}$ is updated by $\boldsymbol{Q}$ (lines 20-23).

In addition, $\boldsymbol{Q}$ can also be derived using the exponentially weighted moving average (EWMA) method, especially in complex environments, such as those suffering from serious fading, mobility, and interference by other technologies, etc.

4.3 Performance-bounded Strict Channel Scheduling

For the channel scheduling optimization problem (6)-(9), this section first analyzes its computational complexity, describes the shortcomings of the classic solutions, and followed by a greedy solution with performance bounds.

Theorem 1. *The decision problem corresponding to the combinatorial optimization defined in (6)-(9) is NP-hard.*

Proof. The problem of selecting the max number of collision-free events (e.g., n_x) from n events is a case of optimization (6)-(9) under $m = 1$. This problem is reduced by maximum independent set (MIS) problem of graph G in poly-time.

The transformation is as follows: (a) For each vertex $v_i \in G$, $\forall i \in \mathbb{Z}_n$, construct an event e_i such that $s_i = 1$, $f_i = 2$. (b) For any $v_i, v_j \in G$, if there exists an edge (v_i, v_j), then $c_{ij} = 1$, else $c_{ij} = 0$. Then, it holds: there exists n_x collision-free events iff G has an independent set of n_x vertices. □

It is challenging to find exact poly-time solutions for the optimization (6)-(9) in light of Theorem 1. The intuitive idea is to employ search-based methods, such as branch-and-bound [33], binary-search [34], to find a (near) optimal solution. Further, we leverage semidefinite programming based methods, such as [35], [36], to achieve a proper approximate solution. However, the extensive experiments imply that these methods introduce huge search space, heavy computation overhead, and their perform efficiency affects the network scalability, since time-consuming solutions may reduce CCI downlink opportunities (detailed in Section 6.1).

Given these points, we propose a greedy based algorithm, which selects as many collision-free events as possible for each of m groups, and assigns a unique frequency channel for each group. It can efficiently obtain a satisfactory solution with a bounded guarantee in most cases, and is suitable for network servers with various computing capabilities.

Specifically, we first construct a G by the following rules: (a) For each event e_i, construct a vertex v_i. (b) Suppose that for any e_i, e_j, $\boldsymbol{x}_i^T \boldsymbol{x}_j = 1$ is satisfied. If $c_{ij} = 0$, insert an edge (v_i, v_j) in G. Then, the problem of finding the maximum collision-free events set for a group is transformed into the problem of finding the maximum clique in G. As is proved, no poly-time algorithm can approximate the maximum clique size within a factor of $n^{1-\epsilon}$ ($\epsilon > 0$), unless $P = NP$ [36]. Thus, we further design a random graphs theory [37] based lightweight scheme in Algorithm 2 to trade-off between the perform efficiency and approximation ratio of the solution. The core idea is to start from the vertex with the largest degree (lines 5-7), and repeatedly choose a vertex with as large degree as possible that is connected to all previously selected vertices (lines 8-12).

Algorithm 2 Strict Channel Scheduling for Single-GW

1: **input:** E, m, $\boldsymbol{R}$
2: **output:** The set of m group events: S
3: With $\boldsymbol{R}$ as an input, construct a graph G from the set E according to the above rules
4: **for** $k \leftarrow 1$ *to* m **do**
5: $v \leftarrow$ the vertex with the largest degree in G
6: $S_k \leftarrow S_k \cup \{v\}$
7: $\Gamma \leftarrow \{$adjacent vertices of $v\}$
8: **while** $\Gamma \neq \varnothing$ **do**
9: $v \leftarrow$ the vertex with the largest degree in Γ
10: $S_k \leftarrow S_k \cup \{v\}$
11: $\Gamma \leftarrow (\Gamma \setminus \{v\}) \cap \{$adjacent vertices of $v\}$
12: **end while**
13: Remove vertex v in G, for any $v \in S_k$
14: **end for**
15: $S \leftarrow \{S_1, S_2, ..., S_m\}$
16: Assign a unique frequency channel for each group in S

Definition 1. $G(n,p)$ is a random graph [37] on n vertices, where each edge appears independently with probability p.
Theorem 2. *Let $p \in (0,1)$ be a constant. With high probability, the above core greedy scheme produces a maximal clique of size $(1-o(1))\log_{1/p} n$ in $G(n,p)$.*
Proof. Available on the Computer Society Digital Library at https://doi.org/10.1109/ICNP55882.2022.9940331. □

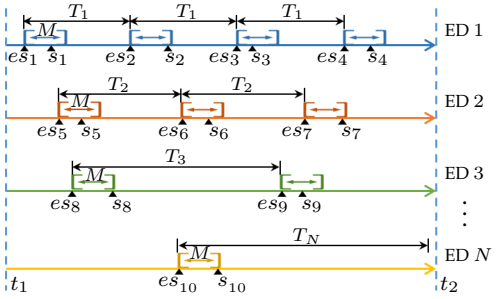


Fig. 4. The scenario of loose periodic transmissions from end-devices.

Proposition 1. *Let E be the set of events, $OPT(E)$ denotes the cardinality of the maximum subset of collision-free events in E. If $\exists U \subseteq E$, such that $\max\{f_i | e_i \in U\} \leq \min\{s_j | e_j \in E \setminus U\}$, then $OPT(E) = OPT(U) + OPT(E \setminus U)$ holds.*
Proof. For any $e_i \in U$, $e_j \in E \setminus U$, it holds that $f_i \leq s_j$ and $c_{ij} = 0$. So $OPT(E)$ is definitely the sum of $OPT(U)$ and $OPT(E \setminus U)$. □

Next, let's analyze the performance and efficiency, resp. Theorem 2 shows the above core greedy scheme can find a maximal clique of size roughly $\log_{1/p} n$ in $G(n,p)$ with high probability. By contrast, it is implied in [37] the size of the largest clique in $G(n,p)$ is roughly $2\log_{1/p} n$ with high probability. Therefore, it is roughly a 2-approximation scheme in most cases. Moreover, the time and space complexity of the algorithm are $O(mn^2)$, $O(n^2)$, resp. To ensure the efficiency in certain extreme cases, Proposition 1 is proposed to speed up the solution by pre-dividing E into multiple subsets.

4.4 Heuristic Loose Channel Scheduling

For some applications where the transmission start times of events are allowed to be offset, this section first formally describes this type of channel scheduling problem, analyzes the computational complexity, and then proposes a heuristic loose channel scheduling strategy.

As shown in Fig. 4, given the maximum offset M, the start time s_i of the event can be selected in the time interval $[es_i, es_i + M]$, where es_i is the earliest transmittable start time. This is termed as "*loose*" accordingly. Accordingly, the objective of this loose channel scheduling problem is to arrange appropriate channel and transmission offset (i.e., $s_i - es_i$, we use notation l_i for simplicity) for each event e_i to maximize the number of successfully delivered packets.
Theorem 3. *The decision problem corresponding to the above loose channel scheduling (LCS) problem is NP-hard.*
Proof. Optimization (6)-(9) is a special case of LCS problem under $M = 0$. By Theorem 1, this result holds. □

Theorem 3 implies the fact that it is highly challenging to obtain an optimal solution with poly-time complexity. To this end, we propose an effective heuristic algorithm, which sets a transmission offset for each event, selects as many collision-free events as possible for each of m groups, and assigns a unique frequency channel for each group.

Specifically, as shown in Algorithm 3, first, for each event in current E, compute the number of collisions between it and any other event in E, and arrange the event with

Algorithm 3 Loose Channel Scheduling for Single-GW

1: **input:** E, m, $\boldsymbol{R}$, M
2: **output:** The set of m group events: S
3: **for** $k \leftarrow 1$ *to* m **do**
4: $U \leftarrow \varnothing$
5: **while** $E \neq \varnothing$ **do**
6: Taking $\boldsymbol{R}$ as an input, for $\forall e_i \in E$, compute its #collisions by $\sum_{j=1}^{|E|} c_{ij}$ under $\boldsymbol{x}_i^T \boldsymbol{x}_j = 1$
7: Find the event e_i in E that has the least collision
8: $S_k \leftarrow S_k \cup \{e_i\}$
9: $E \leftarrow E \setminus \{e_i\}$
10: Taking $\boldsymbol{R}$ as an input, for $\forall e_i \in E$, adjust min l_i, such that for $\forall e_j \in S_k$, $c_{ij} = 0$ under $\boldsymbol{x}_i^T \boldsymbol{x}_j = 1$
11: $U \leftarrow U \cup \{e_i \mid e_i \in E,\ l_i > M\}$
12: $E \leftarrow E \setminus U$
13: **end while**
14: $E \leftarrow U$
15: **end for**
16: $S \leftarrow \{S_1, S_2, ..., S_m\}$
17: Assign a unique frequency channel for each group in S

the least number of collisions to the current group (lines 6-9). Second, for each event in the set of remaining events, adjust the minimum required transmission offset to make it collision-free with all events in this group. If the required offset exceeds the maximum offset M, it is no longer considered in this group (lines 10-12). Repeat these steps until no more events can be added to this group.

The time and space complexity of the above algorithm are $O(mn^3)$ and $O(n)$, respectively.

5 Advanced Design

With the growing scale of applications, LoRa networks may introduce multiple gateways for data aggregation. In a multi-gateway network, a packet transmitted by an end-device only needs to be successfully received by one of the gateways. However, the relationship between the inter-SF interference and successful reception of the corresponding packets in a multi-gateway network is more complicated than that of a single-gateway network, and the channel scheduling problem involved is also more difficult. Thus, this section is dedicated to considering the multi-gateway network scenarios.

5.1 System Model for Multi-GW

Fig. 5 depicts a typical scenario of transmissions from end-devices in a multi-gateway network. Each gateway covers a certain end-devices, and is connected to the same network server via cellular/ethernet. If an end-device is located within the communication range of multiple gateways, the packet transmitted by it is expected to be received by more than one gateway, and the network server is responsible for de-duplicating the packets. Thus, in terms of packet reception, the key difference from the single-gateway networks is that, even if an inter-SF collision occurs, all the concurrently transmitted packets may still finally reach the network server. Since a packet has different signal strengths and transmission interferences at different gateways.

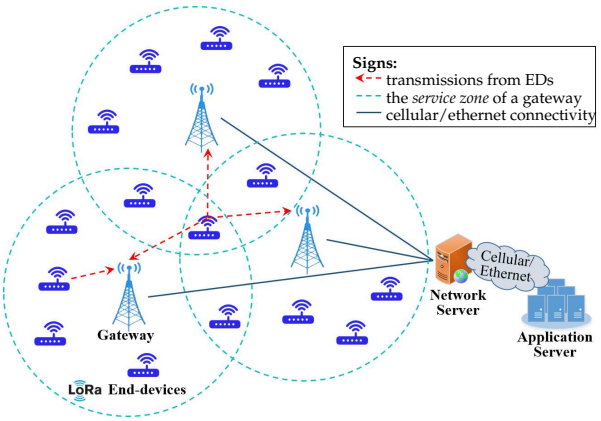


Fig. 5. A typical scenario of transmissions in a multi-gateway network. The *service zone* of a gateway herein refers exclusively to the set of end-devices that can communicate with the gateway.

For further explanation, let's define some notations. GW is the set of gateways in a LoRa network. D^g is the set of all end-devices within the *service zone* of the gateway g. $ED(e_i)$ denotes the transmitter of transmission event e_i. The 0-1 indicator c_{ij}^g in $\boldsymbol{C}^g = [\boldsymbol{c}_1^g\ \boldsymbol{c}_2^g\ ...\ \boldsymbol{c}_n^g] \in \mathbb{R}^{n \times n}$ denotes whether a transmission collision will occur between e_i and e_j, and cause e_i not to be received by gateway g. The 0-1 indicator r_{ij}^g in $\boldsymbol{R}^g = [\boldsymbol{r}_1^g\ \boldsymbol{r}_2^g\ ...\ \boldsymbol{r}_n^g] \in \mathbb{R}^{n \times n}$ denotes whether an inter-SF collision will occur between e_i and e_j, and cause e_i not to be received by gateway g.

Accordingly, to improve the scalability in the multi-gateway network scenarios, we define the following optimization problem by considering imperfect orthogonality.

$$\max_{\boldsymbol{x}_i} \sum_{i=1}^{n} \mathbf{1}^T \boldsymbol{x}_i \Big(1 - \prod_{\forall g \in GW} \big(1 - \prod_{\forall ED(e_j) \in D^g} (1 - c_{ij}^g)\big)\Big) \quad (10)$$

Subject to:

$$\boldsymbol{c}_i^g = \boldsymbol{t}_i \circ (\boldsymbol{X}^T \boldsymbol{x}_i) \circ \boldsymbol{p}_i + \boldsymbol{t}_i \circ (\boldsymbol{X}^T \boldsymbol{x}_i) \circ (\mathbf{1} - \boldsymbol{p}_i) \circ \boldsymbol{r}_i^g \quad (11)$$

$$\mathbf{1}^T \boldsymbol{x}_i \leq 1,\ \forall i \in \mathbb{Z}_n \quad (12)$$

$$x_{ki} \in \{0, 1\},\ \forall i \in \mathbb{Z}_n,\ \forall k \in \mathbb{Z}_m \quad (13)$$

where $\boldsymbol{x}_i$ (defined in Table 1) is the independent variable. $c_{ij}^g = 1$ means e_i cannot be received by gateway g due to the transmission collision between e_i and e_j, and vice versa. $\prod_{\forall ED(e_j) \in D^g} (1 - c_{ij}^g) = 1$ denotes e_i is able to be received by gateway g. $\prod_{\forall g \in GW} \big(1 - \prod_{\forall ED(e_j) \in D^g} (1 - c_{ij}^g)\big) = 1$ denotes e_i cannot be received by any gateways in set GW. Thus, the objective (10) is to assign an appropriate frequency channel for each event, such that the number of packets successfully delivered to the network server is maximized.

To solve this problem, the matrices $\boldsymbol{R}^g, \forall g \in GW$ used in Eq. (11) require to be determined primarily. Thus, we still first design an imperfect-orthogonality aware scheme for multi-gateway networks to detect inter-SF collisions and the reception of corresponding packets in each gateway. Then, an efficient channel scheduling solution for the above optimization is proposed to maximize packet delivery. Besides, additional features are also offered to make it practical.

5.2 Imperfect-Orthogonality Awareness for Multi-GW

To detect inter-SF collisions and the reception of corresponding packets in the *service zone* of each gateway, in-

spired by the design of imperfect-orthogonality awareness for single-gateway networks in Section 4.2, this section proposes an imperfect-orthogonality aware mechanism for multi-gateway network scenarios.

As distinct from single-gateway networks, the goal here is to detect whether an inter-SF collision will occur between two events in the *service zone* of a gateway, and (if occurs) which one(s) cannot be received by this gateway. We similarly observe that this can be achieved by tracking the channel scheduling results of packets and their actual reception in each gateway. The core idea is to check the reception of packets suffering from inter-SF interference in each gateway, e.g., for the case where two end-devices in the *service zone* of a gateway transmit overlapped packets with the same CF and different SFs, if packets transmitted by one of the end-devices are frequently not received by this gateway, it is believed that there is an inter-SF collision that causes packets from this end-device not to be received, and vice versa.

In light of the above idea, we design the imperfect-orthogonality aware scheme for multi-gateway networks, which exploits the channel scheduling results of uplink packets and their actual reception results in each gateway to evaluate whether the gateway can receive the corresponding packets when the transmissions of two end-devices suffer from inter-SF interference (i.e., acquire $\boldsymbol{R}^g$, $\forall g \in GW$). As shown in Algorithm 4, the definitions of the notations E_{ps}, E_{cs}, d, γ are detailed in Section 4.2. $\boldsymbol{Q}^g \in \mathbb{R}^{N\times N}$ indicates the number of inter-SF collisions between end-devices in gateway g, where N is the number of joined end-devices. It is initially a zero matrix and is updated in each round d.

Specifically, with the events that were received by a gateway, update the set of end-devices that each gateway can serve, i.e., update D^g for $\forall g \in GW$ (lines 3-6). Second, for each event e_i sent according to CCI by the end-device in the *service zone* of a gateway but not received by this gateway, find the interfering packet e_j most probably to cause its loss, which provides the most interfering power (lines 11-15), and increase the number of inter-SF collisions for the transmitter of e_i by 1 (lines 16-17). Third, for each event e_i sent by the end-device in the *service zone* of a gateway and received by this gateway, find the interfering packet that has the least impact on its reception, and decrease the number of inter-SF collisions for the transmitter of e_i by 1 (lines 20-23). Finally, $\boldsymbol{R}^g$ is updated by $\boldsymbol{Q}^g$ for $\forall g \in GW$ (lines 25-29).

Algorithm 4 Imperfect-Orthogonality Aware for Multi-GW

1: **input:** E_{ps}, E_{cs}, $(D^g)^d$, $(\boldsymbol{Q}^g)^d$, where $\forall g \in GW$
2: **output:** $(D^g)^{d+1}$, $(\boldsymbol{Q}^g)^{d+1}$, $\boldsymbol{R}^g$, where $\forall g \in GW$
3: **for** each $g \in GW$ **do**
4: **for** each $e_i \in E_{ps}$ **do**
5: **if** gateway g received e_i **then** $D^g \leftarrow D^g \cup ED(e_i)$
6: **end for**
7: $E^g_{ps} \leftarrow \{e_j \mid e_j \in E_{ps}, ED(e_j) \in D^g\}$
8: $E_{dl} \leftarrow \{e_j \mid e_j \in E^g_{ps}, e_j$ was sent according to CCI$\}$
9: $E_{rx} \leftarrow \{e_j \mid e_j \in E^g_{ps}, e_j$ was received by gateway $g\}$
10: **for** each $e_i \in (E_{dl} \setminus E_{rx})$ **do**
11: $E_{cn} \leftarrow \{e_j \mid e_j \in E^g_{ps}, t_{ij} = 1, \boldsymbol{x}_i^T \boldsymbol{x}_j = 1, p_{ij} = 0\}$
12: $e_j \leftarrow$ event with max RSSI in $(E_{cn} \cap E_{rx})$
13: **if** $e_j = \varnothing$ **then**
14: $e_j \leftarrow$event with max history avg. RSSI in $E_{cn} \cap E_{dl}$
15: **end if**
16: Get indexes x, y of $ED(e_i)$, $ED(e_j)$, resp.
17: $q^g_{xy} \leftarrow q^g_{xy} + 1$
18: **end for**
19: **for** each $e_i \in E_{rx}$ **do**
20: $E_{cn} \leftarrow \{e_j \mid e_j \in E^g_{ps}, t_{ij} = 1, \boldsymbol{x}_i^T \boldsymbol{x}_j = 1, p_{ij} = 0\}$
21: $e_j \leftarrow$ event with min RSSI in $(E_{cn} \cap E_{rx})$
22: Get indexes x, y of $ED(e_i)$, $ED(e_j)$, resp.
23: **if** $q^g_{xy} > 0$ **then** $q^g_{xy} \leftarrow q^g_{xy} - 1$
24: **end for**
25: **for** any $e_i,\ e_j \in E_{cs}$ **do**
26: Get indexes x, y of $ED(e_i)$, $ED(e_j)$, resp.
27: **if** $q^g_{xy} \geq \gamma$ **then** $r^g_{ij} \leftarrow 1$ **else** $r^g_{ij} \leftarrow 0$
28: **if** $q^g_{yx} \geq \gamma$ **then** $r^g_{ji} \leftarrow 1$ **else** $r^g_{ji} \leftarrow 0$
29: **end for**
30: **end for**

5.3 Strict Channel Scheduling for Multi-GW

For the channel scheduling optimization problem (10)-(13), this section first analyzes its computational complexity, and then proposes an efficient strict channel scheduling strategy.

Theorem 4. *The decision problem corresponding to the combinatorial optimization defined in (10)-(13) is NP-hard.*

Proof. Optimization (6)-(9) is a special case of optimization (10)-(13) under $|GW| = 1$. By Theorem 1, this result holds. □

Following Theorem 4, it is difficult to find an exact solution in poly-time for the channel scheduling optimization problem (10)-(13). On the other hand, the scale of end-devices and their uplink traffic in multi-gateway networks may increase significantly compared to single-gateway networks. To this end, we design a heuristic channel scheduling strategy to trade-off between the efficiency of the solution and the complexity of the problem.

There is a key observation that packets from different end-devices typically have different probabilities of successful delivery to the network server in a multi-gateway network, since each end-device may be served by a different number of gateways. In view of this feature, the core idea of the designed channel scheduling strategy is to prioritize assigning channels for packets whose transmitters are only served by one gateway, by analogy, assigning channels for packets whose transmitters are served by two gateways, etc. The details are shown in Algorithm 5, where E_z, $z \in \{1, ..., |GW|\}$ denotes the set of events whose transmitters are served by z gateways, i.e., for each event $e_i \in E_z$, $ED(e_i)$ is located within the communication range of z gateways.

Specifically, with D^g updated in each round of Alg. 4, for the events to be transmitted, classify them by the number of gateways that can serve their transmitters (lines 3-7). Second, for the events whose transmitters are only served by one gateway, group them by the gateway that can serve their transmitters, and call Alg. 2 for each group (lines 8-12). Third, for the events whose transmitters are served by two gateways (i.e., E_2), sort them in ascending order of finish time, and try to find a group of events S_k ($k \in \mathbb{Z}_m$) for each $e_i \in E_2$, such that each event in $S_k \cup \{e_i\}$ can be received by ≥ 1 gateway according to Eq. (11) (lines 14-18). By analogy, find a group for each event in E_3 to $E_{|GW|}$ in turn.

The time and space complexity of the above algorithm are $O(mn^2w^2)$ and $O(n^2w)$, respectively, where $w = |GW|$.

Algorithm 5 Strict Channel Scheduling for Multi-GW

1: **input:** E, m, GW, D^g, $\boldsymbol{R}^g$, where $\forall g \in GW$
2: **output:** The set of m group events: S
3: $\{E_1, E_2, ..., E_{|GW|}\} \leftarrow \{\varnothing, \varnothing, ..., \varnothing\}$
4: **for** each $e_i \in E$ **do**
5: $G \leftarrow \{g \mid g \in GW, ED(e_i) \in D^g\}$
6: $E_{|G|} \leftarrow E_{|G|} \cup \{e_i\}$
7: **end for**
8: **for** each $g \in GW$ **do**
9: $E^g \leftarrow \{e_i \mid e_i \in E_1, ED(e_i) \in D^g\}$
10: $\{S_1^g, S_2^g, ..., S_m^g\} \leftarrow$ call Algorithm 2 with E^g, m, $\boldsymbol{R}^g$
11: for $\forall k \in \mathbb{Z}_m$, let $S_k \leftarrow S_k \cup S_k^g$
12: **end for**
13: **for** $z \leftarrow 2$ *to* $|GW|$ **do**
14: sort the events in E_z in ascending order of finish time
15: **for** each $e_i \in E_z$ **do**
16: find min k in $\mathbb{Z}_m$, such that each event in $S_k \cup \{e_i\}$ is received by ≥ 1 gateway according to Eq. (11)
17: **if** k exists **then** $S_k \leftarrow S_k \cup \{e_i\}$ **end if**
18: **end for**
19: **end for**
20: $S \leftarrow \{S_1, S_2, ..., S_m\}$
21: Assign a unique frequency channel for each group in S

Algorithm 6 Loose Channel Scheduling for Multi-GW

1: **input:** E, m, M, GW, D^g, $\boldsymbol{R}^g$, where $\forall g \in GW$
2: **output:** The set of m group events: S
3: $\{E_1, E_2, ..., E_{|GW|}\} \leftarrow \{\varnothing, \varnothing, ..., \varnothing\}$
4: **for** each $e_i \in E$ **do**
5: $G \leftarrow \{g \mid g \in GW, ED(e_i) \in D^g\}$
6: $E_{|G|} \leftarrow E_{|G|} \cup \{e_i\}$
7: **end for**
8: **for** each $g \in GW$ **do**
9: $E^g \leftarrow \{e_i \mid e_i \in E_1, ED(e_i) \in D^g\}$
10: $\{S_1^g, S_2^g, ..., S_m^g\} \leftarrow$ call Alg. 3 with E^g, m, $\boldsymbol{R}^g$, M
11: for $\forall k \in \mathbb{Z}_m$, let $S_k \leftarrow S_k \cup S_k^g$
12: **end for**
13: **for** $z \leftarrow 2$ *to* $|GW|$ **do**
14: sort the events in E_z in ascending order of finish time
15: **for** each $e_i \in E_z$ **do**
16: for $\forall k \in \mathbb{Z}_m$, find a min offset l_{ik} for e_i such that each event in $S_k \cup \{e_i\}$ is received by ≥ 1 gateway
17: $l_i \leftarrow \min(l_{i1}, l_{i2}, ..., l_{im})$, and let k be the index
18: **if** $l_i \leq M$ **then** $S_k \leftarrow S_k \cup \{e_i\}$ **end if**
19: **end for**
20: **end for**
21: $S \leftarrow \{S_1, S_2, ..., S_m\}$
22: Assign a unique frequency channel for each group in S

5.4 Loose Channel Scheduling for Multi-GW

This section also consider the case where the transmission start time of each event is permissibly offset within the transmission period in multi-gateway network scenarios. We first briefly reviews such channel scheduling problem, analyzes its computational complexity, and then designs a channel and transmission offset allocation strategy.

As illustrated in Section 4.4, the notations es_i, s_i are the earliest and real start time of event e_i, resp. $l_i = s_i - es_i$ is the transmission offset of e_i (simply *offset*). Accordingly, the objective of this loose channel scheduling problem for multi-gateway (LCS-MGW) is to arrange appropriate channel and offset for each event, such that the number of packets successfully delivered to the network server is maximized.

Theorem 5. *The decision problem corresponding to the LCS-MGW problem is NP-hard.*

Proof. LCS problem is a special case of LCS-MGW problem under $|GW| = 1$. By Theorem 3, this result holds. □

It follows from Theorem 5 that it is also challenging to find an exact solution in poly-time. Inspired by the observation in Section 5.3, the core idea here is to prioritize assigning channels and offsets for events whose transmitters are only served by one gateway, by analogy, assigning channels and offsets for events whose transmitters are served by two gateways, etc. The details are shown in Algorithm 6.

Specifically, with D^g updated in each round of Alg. 4, for the events to be transmitted, classify them by the number of gateways that can serve their transmitters (lines 3-7). Second, for the events whose transmitters are only served by one gateway, group them by the gateway that can serve their transmitters, and call Alg. 3 for each group (lines 8-12). Third, for the events whose transmitters are served by two gateways (i.e., E_2), sort them in ascending order of finish time, then for each event $e_i \in E_2$, set a minimum offset l_{ik} in each group of events S_k ($\forall k \in \mathbb{Z}_m$), such that each event in $S_k \cup \{e_i\}$ can be received by ≥ 1 gateway according to Eq. (11), and assign e_i to the group with the minimum offset. By analogy, find a group for each event in E_3 to $E_{|GW|}$ in turn.

The time and space complexity of the above algorithm are $O(mn^3w^2)$ and $O(n^2w)$, respectively, where $w = |GW|$.

6 Additional Features

6.1 Closed-loop Adaptive Status Update

To make the design practical, the following critical challenges remain to be addressed. First, the imperfect-orthogonality awareness timing should coordinate with the channel scheduling timing. Second, the channel scheduling timing should adapt to the downlink opportunities of all 3 classes (i.e., A, B, C) of end-devices. Third, the design should support the dynamic join/exit of end-devices, etc.

Thus, we propose a closed-loop adaptive status update mechanism, as shown in Algorithm 7. The notations TS_{ref}, TS_{last} depict the current and last reference timestamps, resp. T_{curr}, T_{next} denote the current and next channel scheduling time duration, resp. $\{T_1, T_2, ...\}$ denote the transmission period set of currently joined end-devices. η is a constant that can impact downlink CCI packet length, CCI downlink request success ratio (the ratio of the successful CCI requests to the total CCI requests), etc. To decide η, we adopt the Monte Carlo method to simulate the downlink CCI distribution process for Class A end-devices. The highest CCI downlink request success ratio is achieved when $\eta = 4$.

The designed mechanism fully considers the downlink communication limitation of Class A end-devices, i.e., the downlink packet is received only in one of the two available windows after an uplink transmission. Moreover, it can adaptively adjust the length of channel scheduling time interval according to the dynamic join/exit of end-devices, without affecting the imperfect-orthogonality awareness.

Algorithm 7 Basic Closed-loop Adaptive Status Update

1: $TS_{last} \leftarrow TS_{ref} \leftarrow$ current reference timestamp
2: $T_{next} \leftarrow T_{curr} \leftarrow$ non-zero initial time duration
3: **while** *true* **do**
4: $T_{next} \leftarrow \eta \max\{T_1, T_2, ...\}$
5: $E_{cs} \leftarrow$predict events in $[TS_{ref}+T_{curr}, TS_{ref}+T_{curr}+T_{next}]$
6: Update $\boldsymbol{R}$ (or $\boldsymbol{R}^g$) by the imperfect-orthogonality awareness for scheduled events E_{ps} in $[TS_{last}, TS_{ref}]$
7: Perform channel scheduling for E_{cs} with $\boldsymbol{R}$ (or $\boldsymbol{R}^g$)
8: Sleep until time $TS_{ref} + T_{curr}$
9: $TS_{last} \leftarrow TS_{ref}$
10: $TS_{ref} \leftarrow TS_{ref} + T_{curr}$
11: $T_{curr} \leftarrow T_{next}$
12: **end while**

6.2 Uplink Prediction and Clock Drift Correction

An end-device provides its transmission period information to the network server when it joins the network, and then periodically transmits uplink packets. The gateway marks each packet just received with the current timestamp and forwards it to the network server. By parsing the packet, the network server obtains the packet counter (from *FCnt* field in LoRaWAN protocol), as well as the corresponding receiving time, etc.

With this information, the network server requires to predict the transmission time of each uplink packet within the next channel scheduling time interval (line 5 in Alg. 7). The intuitive idea is to utilize the transmission period, a packet counter, and the corresponding receiving time to predict the receiving time of any subsequent packets of the end-device. However, due to the clocks of end-devices trend to drift, the prediction may introduce error.

To this end, the same scheme as S-MAC [4] is adopted to correct the clock drift (including phase offset and clock skew) and predict the transmission time of subsequent uplink packets for each end-device. That is, with the relevant information (pairs of the packet counter and the corresponding receiving time) of the recently received uplink packets from an end-device, a least-squares linear regression is performed to estimate the phase offset and compensate for clock skew. Furthermore, considering the prediction accuracy, if a transmission event e_i is predicted to occur in the interval $[s_i, f_i]$, and the error is ε, we update its transmission interval to $[s_i-\varepsilon,\ f_i+\varepsilon]$. A detailed evaluation of the prediction accuracy is provided in Section 7.5.

6.3 Downlink Gateway Selection

The network server requires to send a CCI to an end-device by a gateway to guide its channel selection for subsequent uplink transmissions. In multi-gateway networks, an end-device is possible within the *service zones* of multiple gateways. Thus, the network server should select a proper gateway online for such an end-device when there exists a downlink transmission request (simply *downlink request*).

However, the LoRaWAN protocol does not regulate how to select a gateway among all candidates [38]. The existing studies [38], [39], as well as the practice of network service providers (e.g., TTN, ChirpStack [40]), typically select the gateway with the best link quality based on SNR, RSSI, etc. And yet, they hardly take into account the downlink multi-gateway transmission collisions.

To boost the downlink throughput, we adopt a collision-aware downlink gateway selection policy. Specifically, for a downlink request, check whether it has a co-SF interference with the already scheduled or ongoing downlink requests on any gateway (if exists, deny the request), and find the idle gateways (i.e., have time to perform the downlink request) that are in communication range with the end-device. If multiple gateways match, the network server will select the gateway with the best uplink SNR. Otherwise, the network server does not schedule the downlink request. A detailed evaluation and discussion is provided in Section 7.6.

6.4 Dynamic Join and Exit of End-devices

For network accesses of end-devices, X-MAC supports both modes proposed by LoRaWAN, i.e., *OTAA* and *ABP* [30].

For newly joined end-devices, the network server will dynamically adjust the channel scheduling time duration (Section 6.1), and correct their clock drifts (Section 6.2), so that the newly joined end-devices can get valid CCI steadily.

For the exit of end-devices, the network server traces the uplink and downlink packets of each end-device, and determines an end-device has exited the network by detecting the uplink silence for multiple consecutive transmission periods. Once detected, the network server will remove the exited end-device from the list of joined end-devices, and the channel scheduling time duration is also adaptively adjusted. In addition, to distinguish as much as possible between end-devices leaving the network and end-devices experiencing transmission collisions, the gateway will still forward the corrupted packets to the network server, making it aware that the end-device has not exited the network.

7 Evaluation

7.1 Experiment Setup

Experimental platform: To fully evaluate X-MAC in single-gateway and multi-gateway networks, we developed a real-world hardware testbed and a simulation platform, resp.

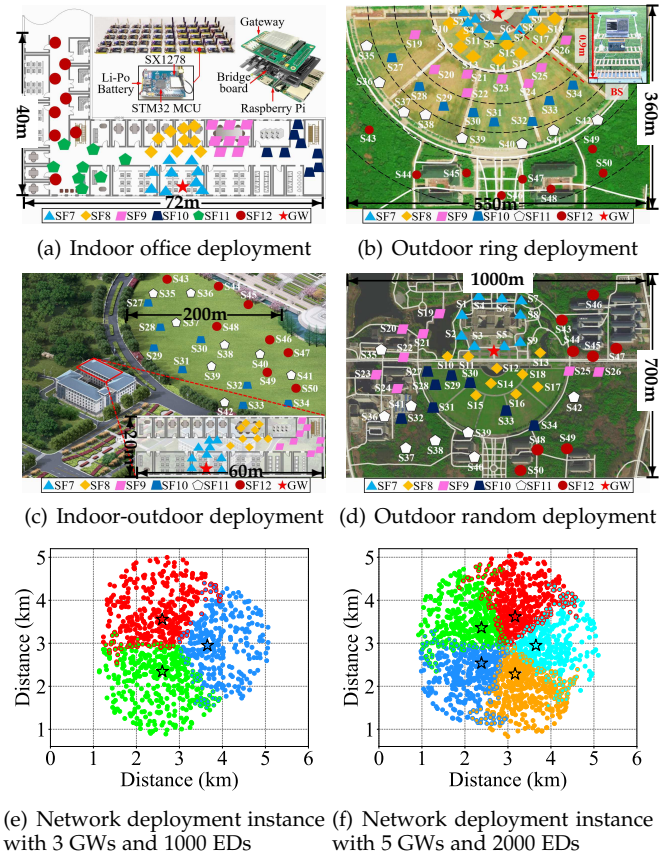
For the **real-world hardware testbed**, it consists of EDs, a GW and a NS. An ED is mainly composed of a microcontroller (MCU) STM32L152, a transceiver chip SX1278 and a Li-Po battery. The full-duplex GW supports 8 parallel uplink channels and 1 downlink channel in CN470-510MHz band, whose hardware architecture is the same as that of S-MAC [4]. Specifically, the uplink employs a SX1301 digital baseband chip and two SX1255 RF front-end chips, the downlink utilizes a SX1278 transceiver chip, and they are controlled by a Raspberry Pi 3 via 2 serial peripheral interfaces. The NS contains 5 threads: *MainThread* is responsible for network status management, operating mode control, and log monitoring, etc.; *UdpRecvThread* mainly receives UDP messages from GWs (e.g., uplink transmission messages, gateway status messages) and replies to the corresponding ACK messages; *PushDataThread* is in charge of parsing uplink packets, correcting EDs' clock offset, and constructing downlink packets, etc.; *RecommendThread* is mainly employed for imperfect-orthogonality awareness, uplink transmission prediction, uplink channel scheduling,

and closed-loop adaptive status update; *PullRespThread* is primarily used for downlink gateway selection, and downlink transmission initiation. The NS is deployed on a laptop with an Intel Core i5 CPU and 8 GB RAM.

For the **simulation platform**, it is a custom discrete-event simulator based on SimPy [41]. The core components in SimPy include the *Events*, *Processes*, and *Environment*. *Events* are used to describe any kind of events. They are tightly bound to time, and scheduled at a given simulation time. *Processes* (defined by Python generator functions) implement the simulation model, that is, they define the behavior of the simulation, and yield instances of *Event*. *Environment* stores these events in its event list and keeps track of the current simulation time. When a *process* yields an *event* (e.g., *Timeout event*), the *process* gets suspended. SimPy resumes the *process* when the *event* occurs. Accordingly, given that the core functions of X-MAC are uplink transmission, downlink CCI transmission, imperfect-orthogonality awareness and channel scheduling, this simulation platform designs three *processing functions*: uplink transmission of EDs, downlink transmission of GWs, and closed-loop adaptive status update of NS. It enables LoRa network simulation by modeling EDs, GWs, and NS, and implements Class A features of the LoRaWAN protocol. The simulator holds a realistic LoRa transmission collision model, which includes co-SF and inter-SF collision. Accordingly, the thresholds for both co-SF and inter-SF collision are obtained from the literature [13], [18]. The adopted log-distance path loss model uses the parameters defined in [19]. In addition, in the simulation platform, to simulate phase offset, each end-device's clock is assigned a fixed random clock shift within ± 20 ppm, since the real-time clock (RTC) clock source in real-world hardware end-devices has a frequency tolerance of ± 20 ppm. To simulate clock skew, a random error that follows a uniform distribution on the interval $[-2$ ms, 2 ms$]$ is introduced for each transmission start time of an end-device, where parameters of the uniform distribution are determined from the results of Fig. 18 in Section 7.5.

Scenarios: For the **single-gateway network scenarios**, we deploy the real-world testbed containing 50 EDs and 1 GW in the indoor office (Fig. 6(a)), outdoor university campus (Fig. 6(b), 6(d)), and mixed indoor-outdoor (Fig. 6(c)), resp. In each scenario, the number of EDs using SF7 to SF12 is 9, 9, 8, 8, 8, and 8, resp. EDs with small SF are deployed closer to the GW, and EDs with large SF are farther away. EDs with the same SF are randomly deployed within a certain spatial range. All EDs boot up randomly, periodically transmit unconfirmed packets, and operate as Class A to evaluate the lower-bound performance since it has the fewest downlink opportunities. The bandwidth (BW), coding rate (CR), packet length, and transmission power (TP) default to 125 kHz, 4/5, 40 bytes, and 14 dBm, resp. Each experiment under each configuration lasts ≥ 120 minutes and is conducted 5 times in different times of the day over 5 months.

For the **multi-gateway network scenarios**, we deploy multiple GWs uniformly on a circle (i.e., any two GWs have the same angle to the center of the circle), randomly choose a location for each ED, and set the minimum SF for the ED, such that it is within the communication range of ≥ 1 GW. Since this SF setting method is the design goal of the current adaptive data rate (ADR) mechanism, and is also adopted in many LoRaWAN deployments [42], making it typical in practical scenarios. Moreover, it minimizes the transmission collision and interference range of each ED, thus enabling to evaluate the lower-bound performance of X-MAC. Fig. 6(e), 6(f) show deployment instances of a 3-gateway network and a 5-gateway network, resp. The operation mode (Class A), transmission feature (periodicity) and parameters (BW, CR, packet length, etc.) of all EDs are consistent with those in the single-gateway networks. A random delay is performed when an end-device starts, followed by the first uplink transmission. In addition, we employ only one unique downlink CF channel in any multi-gateway network to evaluate the lower-bound performance, since this case leads to the most downlink collisions.



(a) Indoor office deployment (b) Outdoor ring deployment

(c) Indoor-outdoor deployment (d) Outdoor random deployment

(e) Network deployment instance with 3 GWs and 1000 EDs (f) Network deployment instance with 5 GWs and 2000 EDs

Fig. 6. Multiple experimental fields, where (a)-(d) are all single-gateway networks based on the real-world testbed, (e)-(f) are all multi-gateway networks based on the simulation platform. The pentagram in (e)-(f) denotes a GW, a filled circle denotes an ED, a single-colored ED is within the communication range of the same colored GW, and a multi-colored ED is within the communication range of corresponding colored GWs.

Method: For the single-gateway networks based on the **real-world testbed**, considering that evaluating the scalability of large-scale LoRa networks in practice is challenging as the manufacture, configuration, deployment, testing, and maintenance will be prohibitively expensive [19], the method widely accepted by existing research is to emulate a larger number of EDs by altering the transmission periods of real-world EDs [4], [5], [20], and we also adopt this method.

Specifically, we treat an ED transmitting every 15 minutes as a standard transmission period (relative reference value only, other values are fine). Thus, a real-world ED that transmits ℓ ($\ell > 0$) times periodically in 15 minutes,

emulates ℓ EDs. For instance, if the transmission period of 50 real-world EDs is set to 5 minutes, it means the relative number of concurrent EDs is 150 (15min/5min*50EDs). Thus, we use *relative end-devices* to denote the relative number of concurrent EDs, which in fact corresponds to a transmission period of 50 real-world EDs.

For the multi-gateway networks based on the **simulation platform**, considering that a multi-gateway network typically covers a larger range than a single-gateway network, the above testbed containing 50 real-world EDs is not sufficient to evaluate the performance of X-MAC in multi-gateway networks. Thus, recent studies evaluating multi-gateway networks are typically done via simulators [10], [14], [18], and we also take this approach.

Specifically, we set the transmission period of each ED to 1 minute in our simulation platform. Given the number of GWs and locations, the performance of X-MAC is evaluated by varying the number of randomly deployed EDs. Each experiment under each configuration runs for 5 days of simulated time, and 20 independent replications are performed.

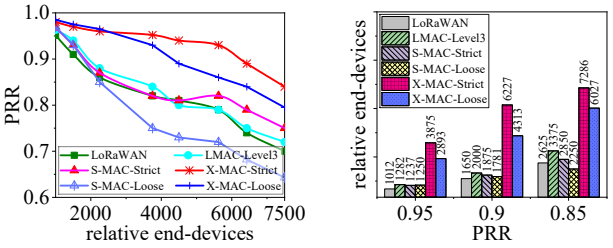
Baseline: X-MAC is compared with the following state-of-the-art methods:

- LoRaWAN [22]: an unslotted ALOHA based method, where an ED randomly selects a CF channel from a list of available channels for each transmission.
- S-MAC [4]: a pure channel scheduling based method. Similar to X-MAC, it also contains both strict and loose models for varying demands. Limited by transmission periods of real-world EDs, M is set to 4 s.
- LMAC [5]: a channel activity detection based method. The best configuration "Level 3" provided by LMAC is employed. Each packet is only allowed to be transmitted in its own transmission period.

7.2 PRR Performance of X-MAC

7.2.1 PRR in the indoor office scenario

We conduct an indoor experiment shown in Fig. 6(a) to evaluate the packet reception rate (PRR, the ratio of correctly received de-duplicated packets by the NS to total transmitted packets) of X-MAC with the increase of EDs.



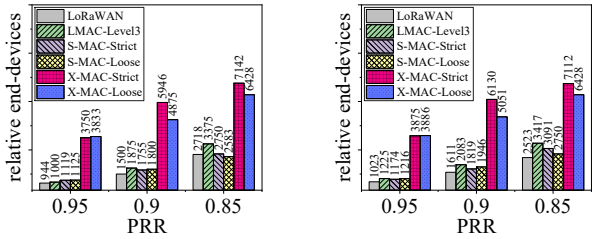
(a) PRR vs. relative end-devices (b) relative end-devices vs. PRRs

Fig. 7. PRR of X-MAC in the indoor scenario.

As shown in Fig. 7(a), X-MAC achieves a significant PRR performance boost compared with the state-of-the-art methods. Furthermore, Fig. 7(b) indicates the relative number of end-devices supported under different PRRs of 95%, 90%, and 85%. Compared with the state-of-the-art methods, X-MAC brings up the number of concurrent end-devices by 2.02× with PRR requirement of $>$ 95%, and by 2.11× with PRR requirement of $>$ 90%.

7.2.2 PRR in the outdoor scenarios

We conduct outdoor experiments with the scenarios in Fig. 6(b) and 6(d), respectively, to evaluate the PRR of X-MAC. As shown in Fig. 8, with PRR requirement of $>$ 95%, X-MAC increases the number of concurrent end-devices by 2.41× compared with the state-of-the-art methods, and 2.97× compared with LoRaWAN in the ring deployment scenario. X-MAC boosts the number of concurrent end-devices by 2.17× compared with the state-of-the-art methods, and 2.79× compared with LoRaWAN in the random deployment scenario.

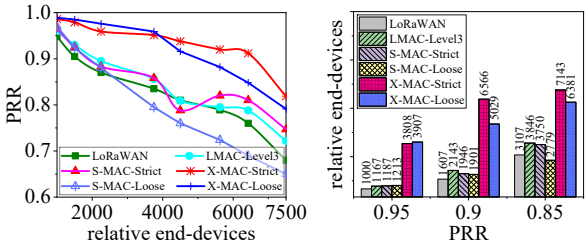


(a) Outdoor ring deployment (b) Outdoor random deployment

Fig. 8. PRR of X-MAC in the outdoor scenarios.

7.2.3 PRR in the mixed indoor-outdoor scenario

We also conduct an experiment shown in Fig. 6(c) to evaluate the PRR of X-MAC. As illustrated in Fig. 9, with PRR requirement of $>$ 95%, X-MAC brings up the number of concurrent end-devices by 2.22× compared with the state-of-the-art methods, and 2.81× compared with LoRaWAN. With PRR requirement of $>$ 90%, X-MAC boosts the number of concurrent end-devices by 2.06× compared with the state-of-the-art methods, and 3.08× compared with LoRaWAN.



(a) PRR vs. relative end-devices (b) relative end-devices vs. PRRs

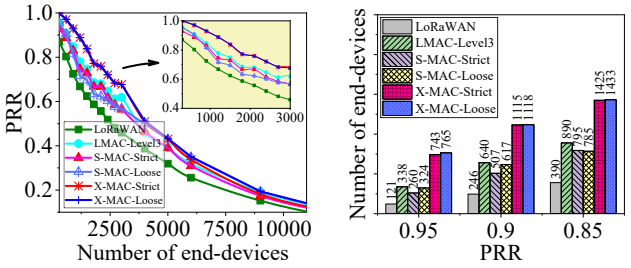
Fig. 9. PRR of X-MAC in the mixed indoor-outdoor scenario.

7.2.4 PRR in 3-gateway network scenarios

We then conduct an experiment in 3-gateway networks with different numbers of EDs to evaluate PRR of X-MAC, and Fig. 6(e) shows one of the deployment examples. The result is given in Fig. 10. It is observed that X-MAC supports 765 EDs for concurrent transmission with PRR requirement of $>$ 95%, increasing the number of concurrent EDs by 1.26× compared with the state-of-the-art methods, while supports 1,115 and 1,433 EDs for concurrent transmission with PRR requirements of $>$ 90% and 85%, respectively.

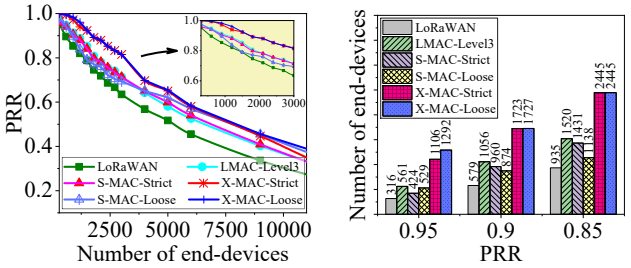
7.2.5 PRR in 5-gateway network scenarios

We further conduct an experiment in 5-gateway networks with different numbers of EDs to evaluate PRR of X-MAC, and Fig. 6(f) shows a deployment example. The result is presented in Fig. 11. We observe that X-MAC supports up to 1,292 EDs for concurrent transmission with PRR requirement of $>$ 95%, bringing up the number of concurrent EDs by 1.30× compared with the state-of-the-art methods, while



(a) PRR vs. number of EDs (b) Number of supported EDs

Fig. 10. PRR of X-MAC in 3-gateway network scenarios.



(a) PRR vs. number of EDs (b) Number of supported EDs

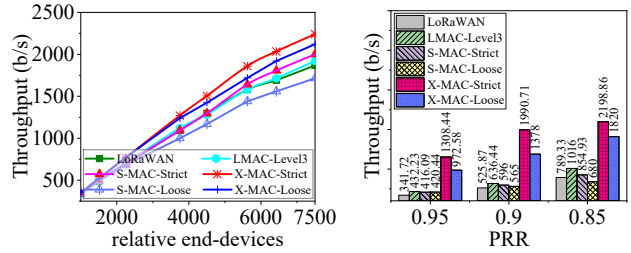
Fig. 11. PRR of X-MAC in 5-gateway network scenarios.

supports 1,727 and 2,445 EDs for concurrent transmission with PRR requirements of $>$ 90% and 85%, respectively.

To sum up, the PRR results in Fig. 7 - 11 highlight the following observations. First, the PRRs of S-MAC and LMAC degrade drastically in these multi-SFs decentralized deployed scenarios. Second, X-MAC achieves superior PRR performance in both single-gateway and multi-gateway networks. Third, the PRR of X-MAC-Loose is not always better than that of X-MAC-Strict in single-gateway networks. The root is that the downlink CCI distribution efficiency in X-MAC-Loose is weaker than that in X-MAC-Strict. Specifically, it is first confirmed that the performance of the loose channel scheduling algorithm always beats the corresponding strict channel scheduling algorithm (Section 7.4). However, compared to X-MAC-Strict, X-MAC-Loose needs to transmit more information (i.e., channel and transmission offset) in each downlink packet to each end-device. It is expected to be further improved by reducing the resolution of transmission offset, compressing downlink packets, etc.

7.3 Throughput Performance of X-MAC

7.3.1 Throughput in the indoor office scenario



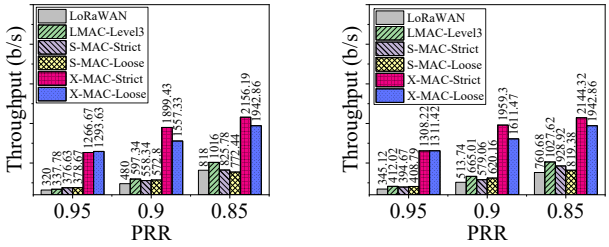
(a) Throughput vs. relative EDs (b) Throughput vs. PRRs

Fig. 12. Throughput of X-MAC in the indoor scenario.

We evaluate the throughput (defined as the total payloads correctly received and de-duplicated by the NS per unit of time) of X-MAC with the scenario in Fig. 6(a). As shown in Fig. 12(a), X-MAC achieves the best throughput performance. Moreover, the throughput under PRRs of 95%, 90%, and 85% is given in Fig. 12(b). With PRR requirement of $>$ 95%, the throughput supported by X-MAC reaches 1308.44 bps, bringing up to 2.03$\times$ improvement compared with the state-of-the-art methods. With PRR requirement of $>$ 90%, the throughput of X-MAC reaches 1990.71 bps, boosts 2.13$\times$ improvement compared with the state-of-the-art methods.

7.3.2 Throughput in the outdoor scenarios

We evaluate the throughput of X-MAC with the scenarios in Fig. 6(b) and 6(d), resp. As shown in Fig. 13, X-MAC also wins the best network throughput performance. With PRR requirement of $>$ 95%, X-MAC achieves $\sim$ 1.3 Kbps network throughput in both outdoor ring deployment and random deployment scenarios, and boosts 2.42$\times$ and 2.18$\times$ improvement compared with the state-of-the-art methods, respectively. With PRR requirement of $>$ 90%, the throughput is up to $\sim$ 1.9 Kbps in both outdoor scenarios, bringing 2.18$\times$ and 1.95$\times$ performance improvement compared with the state-of-the-art methods, respectively.

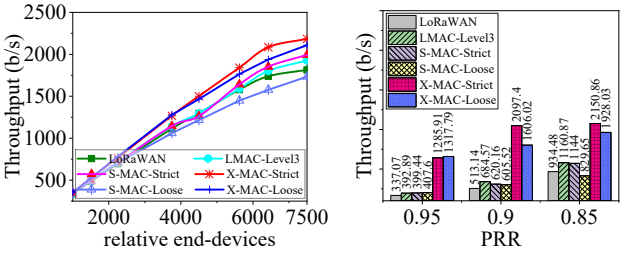


(a) Outdoor ring deployment (b) Outdoor random deployment

Fig. 13. Throughput of X-MAC in the outdoor scenarios.

7.3.3 Throughput in the mixed indoor-outdoor scenario

We also evaluate the throughput of X-MAC with the scenario in Fig. 6(c). The results are illustrated in Fig. 14. With PRR requirement of $>$ 95%, X-MAC boosts the throughput by 2.23$\times$ compared with the state-of-the-art methods. With PRR requirement of $>$ 90%, the throughput of X-MAC reaches $\sim$ 2.1 Kbps, bringing 2.06$\times$ performance improvement compared with the state-of-the-art methods.



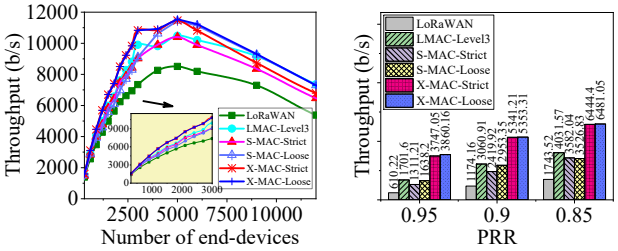
(a) Throughput vs. relative EDs (b) Throughput vs. PRRs

Fig. 14. Throughput of X-MAC in the mixed indoor-outdoor scenario.

7.3.4 Throughput in 3-gateway network scenarios

We then evaluate the throughput of X-MAC in 3-gateway networks with different numbers of EDs, and Fig. 6(e) is a deployment example. As presented in Fig. 15, the throughput performance shows a trend of increasing and then decreasing with the increase of end-devices in the network. With PRR requirement of $>$ 95%, X-MAC improves the

throughput by up to 1.27× compared with the state-of-the-art methods. X-MAC achieves ∼ 3.9 Kbps, 5.4 Kbps and 6.5 Kbps network throughput with PRR requirements of > 95%, 90%, and 85%, respectively.

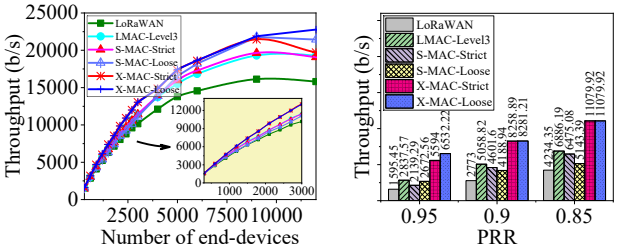


(a) Throughput vs. number of EDs (b) Throughput vs. PRRs

Fig. 15. Throughput of X-MAC in 3-gateway scenarios.

7.3.5 *Throughput in 5-gateway network scenarios*

We further evaluate the throughput of X-MAC in 5-gateway networks with different numbers of EDs, and Fig. 6(f) is a deployment example. As shown in Fig. 16, X-MAC exhibits even better network throughput performance in the 5-gateway network scenarios. With PRR requirement of > 95%, X-MAC boosts the throughput by up to 1.30× compared with the state-of-the-art methods. X-MAC achieves ∼ 6.5 Kbps, 8.3 Kbps and 11.1 Kbps network throughput with PRR requirements of > 95%, 90%, and 85%, respectively.

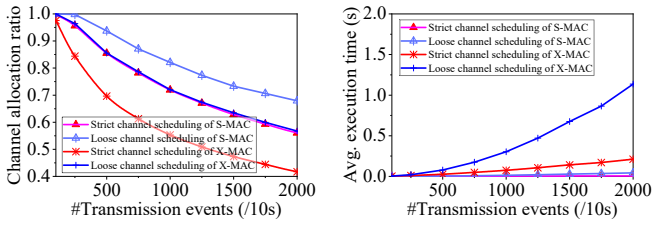


(a) Throughput vs. number of EDs (b) Throughput vs. PRRs

Fig. 16. Throughput of X-MAC in 5-gateway scenarios.

7.4 Performance and Efficiency of Channel Scheduling

To evaluate the channel scheduling algorithms, we randomly generate different numbers of events in 10 s interval, perform each algorithm in turn, and measure their channel allocation ratio and the corresponding avg. execution time.



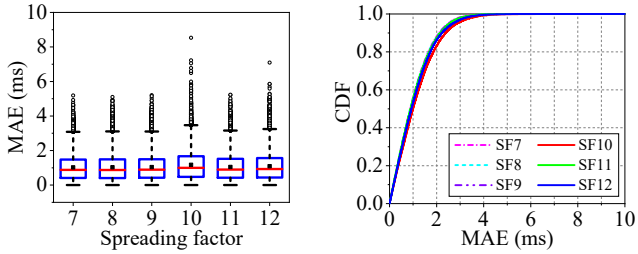
(a) Performance of algorithms (b) Efficiency of algorithms

Fig. 17. Performance and efficiency of channel scheduling algorithms.

As shown in Fig. 17(a), the channel scheduling algorithms' performance of X-MAC is worse than that of S-MAC, but the PRR and throughput of X-MAC evaluated earlier are much better than that of S-MAC. Thus, it can be seen that the imperfect-orthogonality aware scheme of X-MAC makes a great contribution to improving network scalability. Moreover, it is observed that the performance of the loose channel scheduling algorithm always beats the corresponding strict channel scheduling algorithm. Fig. 17(b) indicates the efficiency of strict channel scheduling algorithm in X-MAC is very close to that of S-MAC, and the efficiency of loose channel scheduling algorithm in X-MAC is also acceptable in most scenarios.

7.5 Prediction Accuracy of Uplink Transmissions

To evaluate the prediction accuracy for the transmission time of uplink packets, we take the predicted and actual receiving time data of uplink packets from evaluation 7.8, and measure using the mean absolute error (MAE) with formula $\sum_{i\in n}|y_i-\hat{y}_i|/n$, where $\hat{y}_i$ and y_i is the predicted and actual receiving time of packet i, respectively. Fig. 18(a) shows that the average prediction error of uplink packets is ∼ 1 ms. Fig. 18(b) illustrates that the prediction errors of uplink packets under different SFs are very similar, and the probabilities of prediction error within 2 ms and 3 ms are above 83% and 95%, respectively.

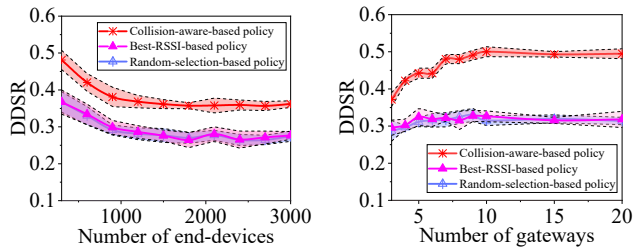


(a) Prediction error distribution (b) CDF of the prediction error

Fig. 18. Prediction error and error probability distribution.

7.6 Performance of Downlink Gateway Selection

To evaluate the downlink performance of the collision-aware downlink gateway selection policy, we use the downlink delivery success ratio (DDSR) as a metric. It is the ratio of the successful downlink requests to the total requests. As stated in Section 7.1, we first deploy 3 GWs uniformly on a circle and randomly deploy different numbers of EDs, to clarify the impact of gateway selection policies on the DDSR. Second, we limit the number of deployed EDs to 1000, deploy varying numbers of GWs uniformly on a circle, and investigate the DDSR under different numbers of GWs. We compare the collision-aware gateway selection policy with existing best-RSSI-based and random-selection-based policies. The results are presented in Fig. 19(a) and 19(b), resp. It shows that our collision-aware-based policy improves the downlink delivery success ratio by ∼ 26% to 56% compared with the existing methods.



(a) DDSR vs. number of EDs (b) DDSR vs. number of GWs

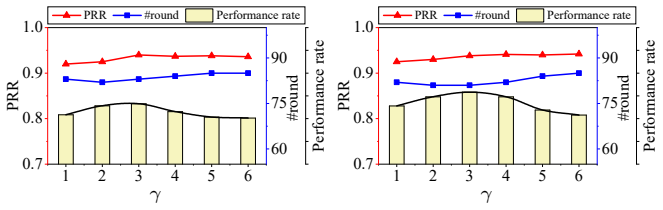
Fig. 19. Performance of collision-aware downlink gateway selection.

In addition, the number of GWs in a LoRa network tends to be small relative to the number of EDs, and the inter-SF

collisions caused by concurrent downlink transmissions of GWs tend to be small. Thus, the collision-aware gateway selection policy does not consider being aware of downlink inter-SF collisions of multiple GWs. Naturally, we can similarly employ the proposed imperfect-orthogonality aware scheme to detect and avoid downlink inter-SF collisions.

7.7 Inter-SF Collision Threshold Setup of X-MAC

We perform indoor and mixed indoor-outdoor experiments shown in Fig. 6(a) and 6(c) to determine the inter-SF collision threshold γ in X-MAC. The transmission period for all 50 end-devices is set to 10 s. The γ affects PRR and inter-SF collision detection efficiency, and Fig. 20 shows the results of these metrics under various γ. The collision detection efficiency is defined by "#round", which denotes the scheduling rounds required to receive a total of N_p packets ($N_p = 15,000$ here, other large values are acceptable). The "performance rate" is the ratio of PRR to the required #round. We observe that PRR is stable when $\gamma \geq 3$, and the best performance ratio is achieved with $\gamma = 3$.

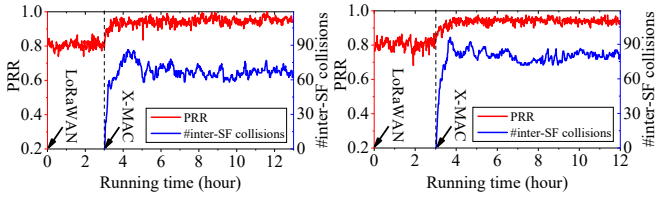


(a) Indoor office deployment (b) Indoor-outdoor deployment

Fig. 20. Performance vs. differnet γ in two scenarios.

7.8 Stability and Reliability of X-MAC

We then further conduct indoor and mixed indoor-outdoor experiments shown in Fig. 6(a) and 6(c) respectively to evaluate the stability and reliability of X-MAC under $\gamma = 3$. The transmission period for all 50 end-devices is also set to 10 s. The network server runs the native LoRaWAN for 3 hours, and then starts X-MAC. As shown in Fig. 21, the system rapidly detects the inter-SF collisions after X-MAC is started, and the PRR promptly boosts from $\sim$ 80% to 94%. Also, the system offers a fast convergence time of 15 to 30 minutes in terms of PRR performance. In addition, with the change of time and environment, the detected inter-SF collisions fluctuate slightly, while the PRR remains stable.

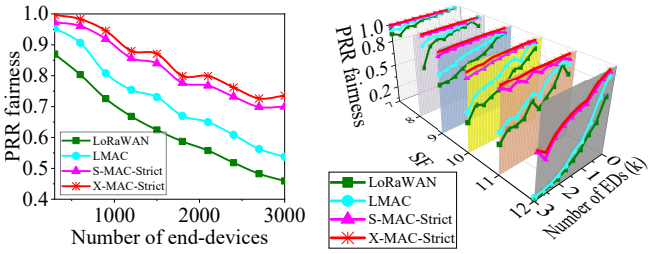


(a) Indoor office deployment (b) Indoor-outdoor deployment

Fig. 21. The stability and reliability of X-MAC.

7.9 PRR Fairness for End-devices

We also evaluate the packet reception rate (PRR) fairness for EDs in 3-gateway networks with different numbers of EDs. The PRR fairness for EDs is measured by Jain' fairness index with formula $(\sum_{id=1}^{N} PRR_{id})^2/(N\sum_{id=1}^{N} PRR_{id}^2)$, where PRR_{id} denotes the uplink PRR of ED id, N is the total



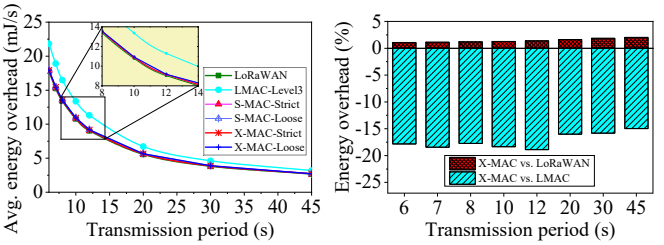
(a) PRR fairness for overall EDs (b) PRR fairness for classified EDs

Fig. 22. PRR fairness for end-devices in 3-gateway scenarios.

number of EDs. It indicates the similarity of the uplink PRR among different EDs, and ranges from $1/N$ (worst case) to 1 (best case). Fig. 22(a) shows the PRR fairness for overall EDs under different numbers of EDs, and Fig. 22(b) presents the PRR fairness among EDs that use the same SF. It can be see that X-MAC achieves the best PRR fairness compared with the state-of-the-art methods.

7.10 Energy Consumption Overhead of X-MAC

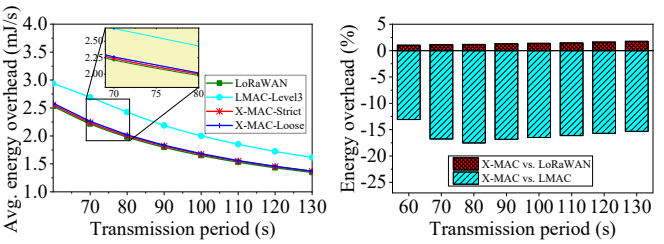
We first conduct experiments with scenarios in Fig. 6(a) and 6(c) respectively to evaluate the energy consumption of transceivers in EDs under various transmission periods. A portable power monitor with μA-resolution is employed to estimate the average energy overhead.



(a) Energy vs. transmission period (b) X-MAC vs. LoRaWAN, LMAC

Fig. 23. Energy consumption of transceivers in indoor-outdoor scenario.

Fig. 23 shows the energy consumption results of mixed indoor-outdoor scenario (due to similar results and limited space, the figure of another scenario is omitted). We observe that X-MAC only increases the energy consumption by $\sim$ 1% to 2% compared with LoRaWAN, while it decreases by $\sim$ 15% to 19% compared with LMAC. In addition, an end-device can employ a ring buffer with tens of bytes to access the CCI, which can also be met by almost all MCUs.



(a) Energy vs. transmission period (b) X-MAC vs. LoRaWAN, LMAC

Fig. 24. Energy consumption of transceivers in 3-GW network scenario.

We further evaluate the energy consumption of transceivers in EDs in a multi-gateway network. The network includes 3 GWs deployed in different locations and 1000 randomly deployed EDs. An energy consumption

monitor is built in the simulation platform, which takes the typical power information of the transceiver collected by the above power monitor in various states to evaluate the energy overhead of the transceiver. As shown in Fig. 24, X-MAC consumes $\sim$ 1% to 2% more energy than LoRaWAN, and consumes $\sim$ 15% to 17% less energy than LMAC.

8 Related Work

To achieve high scalability for LoRa networks, researchers introduce various approaches to resolve or avoid collisions.

PHY-layer approaches. Eletreby *et al.* [1] exploited nodes' natural hardware offsets to resolve collisions. Xia *et al.* [2] considered the time and frequency domain features to separate collided packets. Xia *et al.* [17] employed multiple antennas of a gateway to scale the concurrent transmissions. Tong *et al.* [3], [43] and Xu *et al.* [44] utilized subtle packet time offset to decompose multiple packets. Shahid *et al.* [45] presented a concurrent interference cancellation technique that cancels out interfering symbols by selecting the optimal sub-symbol set. Garlisi *et al.* [46] designed a successive interference cancellation method by synchronizing superposed signals and adding a clock tracking scheme during packet reception. Zhang *et al.* [47] proposed an interference cancellation method based on the segmentation of received signal. These works can resolve collisions and support more concurrent transmissions, but they require direct access to PHY samples and are not feasible on commodity chips.

MAC-layer approaches. Gamage *et al.* [5] avoided transmission collision by detecting whether the specified CF/SF channel was idle. It mitigated collisions and balanced loads of channels, but it failed to detect transmissions with different SFs, which still met severe packet loss in practical LoRa networks. Xu *et al.* [4] utilized pure channel scheduling to avoid collisions. It improved the scalability in many scenarios, but it did not consider potential collisions among different SFs, which still encountered large collisions in decentralized deployments. Other methods, such as tuning TPs, SFs, CFs, or timeslots [6], [7], [8], [9], [10], [11], [32], performed well in some scenarios, but they may introduce excessive communication overhead, energy consumption, end-devices cost, or hardware complexity.

9 Discussion

Transmission power control scheme. An alternative to solve inter-SF collisions is to tune the TP of EDs close to (or away from) the GW such that the GW receives both transmissions. To achieve this, one way is to employ the channel propagation model and collision condition to decide a TP for each ED. However, it relies heavily on rich prior knowledge and empirical analysis and is easily affected by the ambient environment. The other way is to schedule the TP for each ED. However, according to our experiments in the real environment, it can cause sharp long-term fluctuations in PRR, especially if there are large-scale EDs in the network or EDs join dynamically.

Fast fading resistance. The severe burst drop in signal strength, or a rapid change in channel conditions, may cause packet loss. However, X-MAC can cope with this problem partly. It blames these packet losses on inter-SF collisions and avoids these collisions in the channel scheduling of subsequent events. Furthermore, once the channel conditions are stabilized, this mistake will be corrected by the imperfect-orthogonality awareness. Therefore, X-MAC achieves good resistance to fast fading. The endurance experiments conducted in different scenarios (Section 7.8) achieve stable PRR and #inter-SF collision, which also partly confirms this.

10 Conclusion

In this work, we present X-MAC, the first scheduler that considers imperfect-orthogonality in practical LoRa networks. We overcome several practical challenges in X-MAC design. We propose the imperfect-orthogonality aware scheme to detect inter-SF collisions. We propose both strict and loose channel scheduling models to maximize packet delivery for meeting the requirements of different scenarios. Further, we offer additional designs, such as closed-loop adaptive status update, clock drift correction, and downlink gateway selection, to enable a practical MAC protocol in LoRa networks. We implement X-MAC on both a real testbed and a simulation platform and thoroughly evaluate its performance in multiple scenarios. The evaluation results show that X-MAC improves the network scalability by 1.26$\times$ to 2.41$\times$ with PRR requirement of $>$ 95% compared with other state-of-the-art methods. Future works will consider achieving high scalability under traffic with properties such as burstiness, self-similarity, etc.

Acknowledgments

This work was supported by the National Key R&D Program of China 2021YFB2900100, the National Natural Science Foundation of China under Grants No. 62172091, 62232004, 61632008, 61602112, the Fundamental Research Funds for the Central Universities, and a grant from the Research Grants Council of the Hong Kong Special Administrative Region, China (Project No. CityU 21216822).

References

[1] R. Eletreby, D. Zhang, S. Kumar, and O. Yağan, "Empowering low-power wide area networks in urban settings," in *Proc. ACM SIGCOMM Conf.* ACM, 2017, pp. 309–321.

[2] X. Xia, Y. Zheng, and T. Gu, "FTrack: parallel decoding for LoRa transmissions," in *Proc. 17th ACM Conf. Embed. Networked Sens. Syst.*, 2019, pp. 192–204.

[3] S. Tong, Z. Xu, and J. Wang, "CoLoRa: Enabling Multi-Packet Reception in LoRa," in *Proc. IEEE INFOCOM*, 2020, pp. 2303–2311.

[4] Z. Xu, J. Luo, Z. Yin, T. He, and F. Dong, "S-MAC: Achieving high scalability via adaptive scheduling in LPWAN," in *Proc. IEEE INFOCOM*, 2020, pp. 506–515.

[5] A. Gamage, J. C. Liando, C. Gu, R. Tan, and M. Li, "LMAC: Efficient carrier-sense multiple access for lora," in *Proc. 26th Annu. Int. Conf. Mobile Comput. Networking*, 2020, pp. 1–13.

[6] B. Reynders, W. Meert, and S. Pollin, "Power and spreading factor control in low power wide area networks," in *Proc. IEEE Int. Conf. Commun.*, 2017, pp. 1–6.

[7] J. Lee, W.-C. Jeong, and B.-C. Choi, "A scheduling algorithm for improving scalability of LoRaWAN," in *Proc. Int. Conf. Inf. Commun. Technol. Convergence*, 2018, pp. 1383–1388.

[8] M. Centenaro and L. Vangelista, "Boosting network capacity in LoRaWAN through time-power multiplexing," in *Proc. 29th IEEE Int. Symp. Person. Indoor Mobile Radio Commun.*, 2018, pp. 1–6.

[9] J. Haxhibeqiri, I. Moerman, and J. Hoebeke, "Low overhead scheduling of LoRa transmissions for improved scalability," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 3097–3109, 2018.
[10] B. Reynders, Q. Wang, P. Tuset-Peiro, X. Vilajosana, and S. Pollin, "Improving reliability and scalability of lorawans through lightweight scheduling," *IEEE Internet Things J.*, vol. 5, no. 3, pp. 1830–1842, 2018.
[11] M. Centenaro and L. Vangelista, "Time-power multiplexing for LoRa-based IoT networks: an effective way to boost LoRaWAN network capacity," *Int. J. Wirel. Inf. Netw.*, vol. 26, no. 4, pp. 308–318, 2019.
[12] B. Reynders and S. Pollin, "Chirp spread spectrum as a modulation technique for long range communication," in *IEEE Symp. Commun. Veh. Technol.*, 2016, pp. 1–5.
[13] D. Croce, M. Gucciardo, S. Mangione, G. Santaromita, and I. Tinnirello, "Impact of LoRa imperfect orthogonality: Analysis of link-level performance," *IEEE Commun. Lett.*, vol. 22, no. 4, pp. 796–799, 2018.
[14] W. Gao, W. Du, Z. Zhao, G. Min, and M. Singhal, "Towards energy-fairness in lora networks," in *Proc. 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 788–798.
[15] A. Balanuta, N. Pereira, S. Kumar, and A. Rowe, "A cloud-optimized link layer for low-power wide-area networks," in *Proc. 18th Int. Conf. Mob. Syst., Appl., Serv.*, 2020, pp. 247–259.
[16] L. Amichi, M. Kaneko, E. H. Fukuda, N. El Rachkidy, and A. Guitton, "Joint allocation strategies of power and spreading factors with imperfect orthogonality in LoRa networks," *IEEE Trans. Commun.*, vol. 68, no. 6, pp. 3750–3765, 2020.
[17] X. Xia, N. Hou, Y. Zheng, and T. Gu, "PCube: scaling LoRa concurrent transmissions with reception diversities," in *Proc. 27th Annu. Int. Conf. Mobile Comput. Networking*, 2021, pp. 670–683.
[18] V. Toro-Betancur, G. Premsankar, M. Slabicki, and M. Di Francesco, "Modeling communication reliability in LoRa networks with device-level accuracy," in *Proc. IEEE INFOCOM*, 2021, pp. 1–10.
[19] M. C. Bor, U. Roedig, T. Voigt, and J. M. Alonso, "Do LoRa low-power wide-area networks scale?" in *Proc. 19th ACM Int. Conf. Model., Anal. Simul. Wirel. Mob. Syst.*, 2016, pp. 59–67.
[20] J. C. Liando, A. Gamage, A. W. Tengourtius, and M. Li, "Known and unknown facts of LoRa: Experiences from a large-scale measurement study," *ACM Trans. Sens. Netw.*, vol. 15, no. 2, p. 16, 2019.
[21] A. M. Yousuf, E. M. Rochester, B. Ousat, and M. Ghaderi, "Throughput, Coverage and Scalability of LoRa LPWAN for Internet of Things," in *Proc. 26th IEEE/ACM Int. Symp. Qual. Serv.*, 2018, pp. 1–10.
[22] LoRa Alliance, "LoRaWAN Specification v1.1," 2021. [Online]. Available: https://lora-alliance.org/resource-hub/lorawan-specification-v1-1
[23] S. Demetri, M. Zúñiga, G. P. Picco, F. Kuipers, L. Bruzzone, and T. Telkamp, "Automated estimation of link quality for LoRa: A remote sensing approach," in *Proc. 18th Inf. Process. Sens. Networks*, 2019, pp. 145–156.
[24] L. Liu, Y. Yao, Z. Cao, and M. Zhang, "Deeplora: Learning accurate path loss model for long distance links in lpwan," in *Proc. IEEE INFOCOM*, 2021.
[25] A. Waret, M. Kaneko, A. Guitton, and N. El Rachkidy, "LoRa throughput analysis with imperfect spreading factor orthogonality," *IEEE Wireless Commun. Lett.*, vol. 8, no. 2, pp. 408–411, 2018.
[26] J. P. S. Sundaram, W. Du, and Z. Zhao, "A survey on lora networking: Research problems, current solutions, and open issues," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 1, pp. 371–388, 2019.
[27] The Things Network, "TTN Homepage," 2022. [Online]. Available: https://www.thethingsnetwork.org
[28] N. Blenn and F. Kuipers, "LoRaWAN in the wild: Measurements from the things network," *arXiv preprint arXiv:1706.03086*, 2017.
[29] A. Rahmadhani and F. Kuipers, "Understanding collisions in a lorawan," *SURF Wiki*, 2017.
[30] LoRa Alliance, "LoRaWAN Regional Parameters v1.1," 2021. [Online]. Available: https://lora-alliance.org/resource-hub/
[31] Semtech, "Semtech SX1302CFDGW1 Introduction," 2022. [Online]. Available: https://www.semtech.com/products/wireless-rf/lora-core/sx1302cfdxxxgw1
[32] M. Hamnache, R. Kacimi, and A.-L. Beylot, "L3sfa: Load shifting strategy for spreading factor allocation in lorawan systems," in *Proc. 45th Conf. Local Comput. Netw.*, 2020, pp. 216–224.
[33] Y. Wang, S. Cai, and M. Yin, "Two efficient local search algorithms for maximum weight clique problem," in *Proc. AAAI Conf. Artif. Intell.*, vol. 30, no. 1, 2016.
[34] C. Lu, J. X. Yu, H. Wei, and Y. Zhang, "Finding the maximum clique in massive graphs," *Proc. VLDB Endow.*, vol. 10, no. 11, pp. 1538–1549, 2017.
[35] N. Bansal, "Approximating independent sets in sparse graphs," in *Proc. 26th Annu. ACM SIAM Symp. Discrete Algorithms*. SIAM, 2014, pp. 1–8.
[36] N. Bansal, A. Gupta, and G. Guruganesh, "On the Lovász theta function for independent sets in sparse graphs," *SIAM J. Comput.*, vol. 47, no. 3, pp. 1039–1055, 2018.
[37] A. Frieze and M. Karoński, *Introduction to random graphs*. Cambridge Univ. Press, 2016.
[38] V. Di Vincenzo, M. Heusse, and B. Tourancheau, "Improving downlink scalability in LoRaWAN," in *Proc. IEEE Int. Conf. Commun.*, 2019, pp. 1–7.
[39] S. Abboud, N. El Rachkidy, A. Guitton, and H. Safa, "Gateway selection for downlink communication in LoRaWAN," in *Proc. IEEE Wireless Commun. Networking Conf.*, 2019, pp. 1–6.
[40] ChirpStack, "Open-source LoRaWAN Network Server," 2022. [Online]. Available: https://www.chirpstack.io
[41] Simpy, "Discrete event simulation for Python," 2022. [Online]. Available: https://simpy.readthedocs.io
[42] D. Garlisi, I. Tinnirello, G. Bianchi, and F. Cuomo, "Capture aware sequential waterfilling for LoRaWAN adaptive data rate," *IEEE Trans. Wireless Commun.*, vol. 20, no. 3, pp. 2019–2033, 2021.
[43] S. Tong, J. Wang, and Y. Liu, "Combating packet collisions using non-stationary signal scaling in LPWANs," in *Proc. 18th Int. Conf. Mob. Syst., Appl., Serv.*, 2020, pp. 234–246.
[44] Z. Xu, P. Xie, and J. Wang, "Pyramid: Real-time lora collision decoding with peak tracking," in *Proc. IEEE INFOCOM*, 2021, pp. 1–9.
[45] M. O. Shahid, M. Philipose, K. Chintalapudi, S. Banerjee, and B. Krishnaswamy, "Concurrent interference cancellation: decoding multi-packet collisions in LoRa," in *Proc. ACM SIGCOMM Conf.*, 2021, pp. 503–515.
[46] D. Garlisi, S. Mangione, F. Giuliano, D. Croce, G. Garbo, and I. Tinnirello, "Interference cancellation for LoRa gateways and impact on network capacity," *IEEE Access*, vol. 9, pp. 128 133–128 146, 2021.
[47] Q. Zhang, I. Bizon, A. Kumar, A. B. Martinez, M. Chafii, and G. Fettweis, "A Novel Approach for Cancelation of Nonaligned Inter Spreading Factor Interference in LoRa Systems," *IEEE Open J. Commun. Soc.*, vol. 3, pp. 718–728, 2022.

Zhuqing Xu received his B.S. degree from HangZhou DianZi University, China, and the M.S. degree from the College of Software Engineering, Southeast University, China. He is currently a Ph.D. candidate at the School of Computer Science and Engineering, Southeast University. His research interests include the Internet of Things, cyber-physical systems, and wireless networks and sensors.

Junzhou Luo received the B.S. degree in applied mathematics and the M.S. and Ph.D. degrees in computer network from Southeast University, Nanjing, China, in 1982, 1992, and 2000, resp. He is a Full Professor with the School of Computer Science and Engineering, Southeast University, Nanjing, China. His current research interests include next generation network architecture, network security, cloud computing, and wireless LAN. Dr. Luo is a member of the IEEE Computer Society and ACM, the Co-Chair of the IEEE SMC Technical Committee on Computer Supported Cooperative Work in Design, and the Chair of ACM SIGCOMM China.

Zhimeng Yin received his Ph.D. degree from University of Minnesota in 2020. He is now an assistant professor with the Department of Computer Science, City University of Hong Kong. His research interests include Internet of Things, mobile computing, and wireless networking.

Shuai Wang received the B.S. and M.S. degrees from the Huazhong University of Science and Technology, China, and the Ph.D. degree from the Department of Computer Science and Engineering, University of Minnesota, in 2017. He is currently a Professor with the School of Computer Science and Engineering, Southeast University. His research interests include the Internet of Things, cyber-physical systems, data science, and wireless networks and sensors.

Ciyuan Chen received her B.S. degree from Nanjing Agricultural University, China. She is currently taking successive master and Ph.D. program at the School of Computer Science and Engineering, Southeast University. Her research interests include Internet of Things, cyber-physical systems, and wireless networks.

Jingkai Lin received the B.S. degree in Software Engineering from Southeast University, Nanjing, China, in 2021. He is currently studying for a master's degree in Computer Science and Technology, Southeast University. His research interests include the internet of Things and wireless networks and sensors.

Runqun Xiong received a Ph.D. degree in computer science from Southeast University. He was with the European Organization for Nuclear Research as a Research Associate for the AMS-02 experiment from 2011 to 2012. He is currently an associate professor at the School of Computer Science and Engineering, Southeast University, China, where he is involved in AMS-02 data processing at the AMS Science Operations Center. His current research interests include cloud computing, Internet of Things, and wireless networks and sensors. He is a member of the ACM and the China Computer Federation.

Tian He (Fellow, IEEE) is currently a Professor with the School of Computer Science and Engineering, Southeast University. He is the author or coauthor of over 300 articles in premier network journals and conferences with over 23,000 citations (H-Index 70). His research interests include wireless networks, networked sensing systems, cyber-physical systems, real-time embedded systems, and distributed systems. He is a fellow of ACM. He was a recipient of the NSF CAREER Award in 2009, the McKnight Land-Grant Chaired Professorship in 2011, the George W. Taylor Distinguished Research Award in 2015, the China NSF Outstanding Overseas Young Researcher I and II in 2012 and 2016, and the eight best paper awards in international conferences, including MobiCom, SenSys, and ICDCS. He has served as the few general/program chair positions for international conferences and on many program committees and also has been an editorial board member for six international journals, including ACM Transactions on Sensor Networks, the IEEE TRANSACTIONS ON COMPUTERS, and IEEE/ACM Transactions on Networking.